

Оглавление MultiAgent (Структура)

- 01. Обзор архитектуры и принципы
- 02. Динамическая система агентов
- 03. Профили агентов
- 04. Фабрика агентов
- 05. Система инструментов
- 06. Определение инструмента
- 07. Менеджер инструментов
- 08. Система памяти (RAG)
- 09. Менеджер памяти
- 10. Определение Workflow
- 11. Движок рабочих процессов
- 12. Расширенный движок процессов
- 13. Событийно-управляемый движок
- 14. Конструктор агентов
- 15. Инструменты конструктора агентов
- 16. Оптимизатор промптов
- 17. Пайплайн Text-to-SQL
- 18. Система плагинов для БД
- 19. Веб-интерфейс / API для Streamlit
- 20. Менеджер агентов для UI
- 21. Менеджер конфигурации
- 22. Единая система логирования
- 23. Система телеметрии
- 24. Система устойчивости
- 25. Механизм повторных попыток
- 26. Circuit Breaker
- 27. Визуализатор HTML
- 28. Менеджер плагинов БД
- 29. Связчик схем
- 30. Модель с повторными попытками
- 31. Система наблюдаемости

Глава 1: Обзор архитектуры и принципы

Эта система — модульная платформа для многоагентных решений. Основная идея: отделить декларативные описания (YAML-профили, workflow) от исполняющей логики (Python-компоненты), чтобы быстро собирать команды агентов под задачу.

Ключевые компоненты

- **DynamicAgentSystem**: дирижёр, который анализирует задачу, подбирает роли и координирует выполнение через manager-агента.
- **Agent Profiles (YAML)**: «должностные инструкции» агентов — роль, инструменты, модель, политика памяти, промпт.
- **AgentFactory**: сборочный цех, превращающий профиль в готового агента (инструменты, память, инструкции).
- **Tool System**: каталог инструментов и менеджер инструментов с телеметрией.
- **Memory (RAG)**: внешняя память на SQLite (тактическая) + ChromaDB (семантическая), единый **MemoryManager**.
- **Workflow Engine**: структурированные процессы по YAML-планам; Enhanced/AI-контроль качества; Event-driven запуск по триггерам.
- **Resilience**: политики повторов, Circuit Breaker, наблюдаемость и логирование.
- **UI/API**: Streamlit-обвязка и **AgentManager** для запуска сценариев без блокировки интерфейса.

Потоки работы

- **Динамический**: «неизвестный путь» → **DynamicAgentSystem** формирует команду на лету, менеджер-агент делегирует задачи.
- **Структурный (Workflow)**: «известный путь» → **WorkflowDefinition** описывает шаги/зависимости; **WorkflowEngine** исполняет; **Enhanced** добавляет планирование/оценку.
- **Событийный**: запуск по событиям/времени/условиям → **Event-Driven Workflow Engine** с триггерами и **Context Enrichment**.

Память и контекст

- Тактическая (SQLite): пошаговые факты, трассировка.
- Семантическая (ChromaDB): поиск релевантного опыта.
- MemoryPolicy: `scope_read` , `search_enabled` , `last_k_steps` , `strategic_write` и др.
- MemoryManager: единая точка записи/чтения и разрешения конфликтов.

Инструменты и наблюдаемость

- YAML-описания инструментов + загрузчик; вызовы через ToolManager.
- Телеметрия (`@with_telemetry`), централизованное логирование и observability.

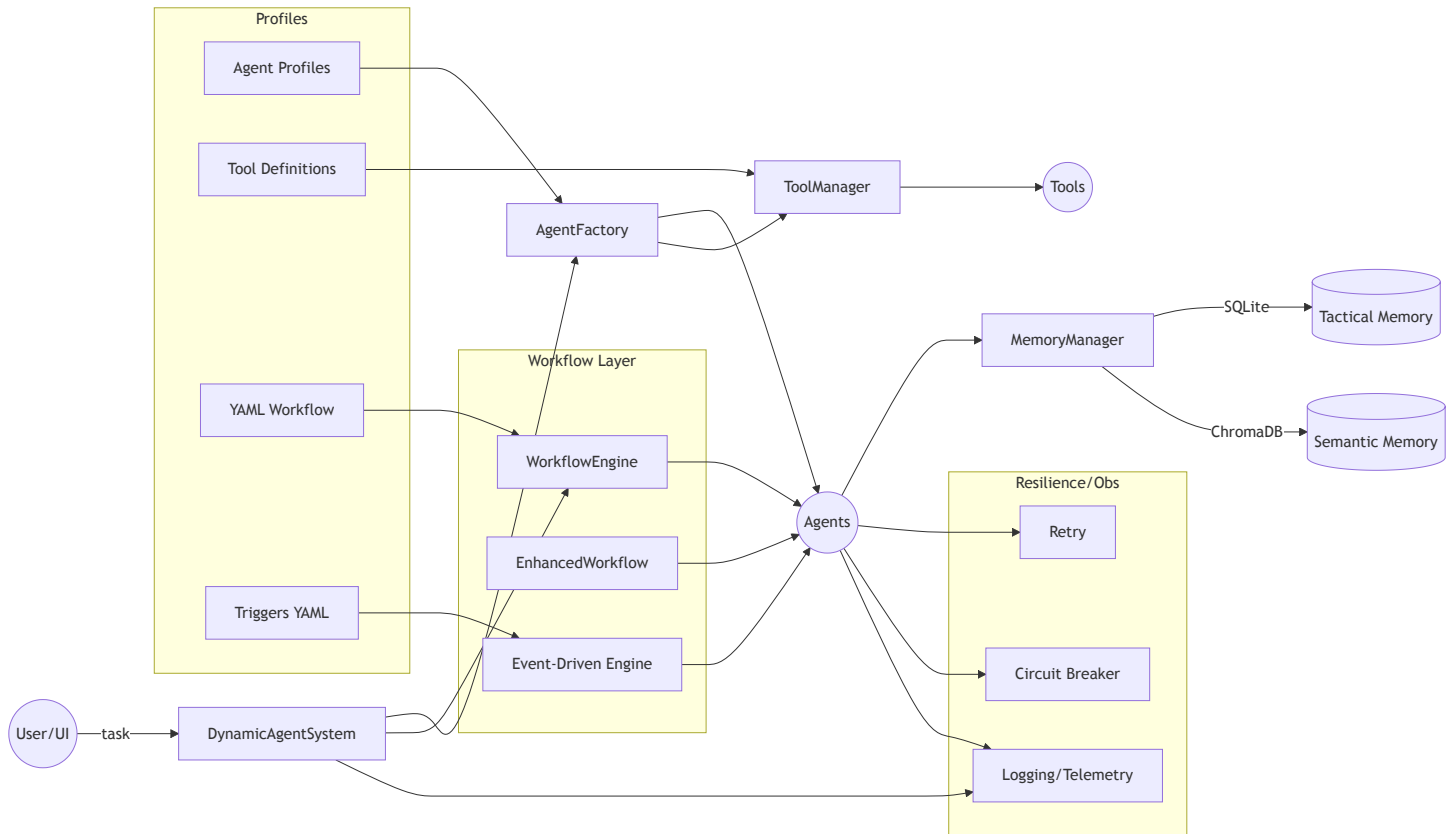
Устойчивость

- Retry Engine с адаптивной стратегией.
- Circuit Breaker для деградации при сбоях.
- Обёртки модели с повторными попытками.

Технологический стек

- YAML для профилей/воркфлоу/триггеров; Python для логики; Streamlit для UI; asyncio/многoproцессность для отзывчивости.

Высокоуровневая схема



Эта архитектура позволяет быстро расширять систему (новые профили/инструменты/воркфлоу), сохраняя предсказуемость, наблюдаемость и устойчивость при работе множества специализированных агентов.

Глава 2: Динамическая система агентов (DynamicAgentSystem)

Когда задача слишком велика для одного исполнителя, нужен оркестр. **DynamicAgentSystem** — это дирижер, менеджер проекта и мозговой центр, который берёт вашу высокоуровневую задачу и сам:

- анализирует, какие роли требуются (LLM-анализ);
- собирает команду через Фабрику Агентов;
- назначает менеджера-агента для координации;
- собирает финальный отчёт.

Зачем это нужно

Один агент быстро упирается в ограничения: смешение ролей, ограниченный контекст, отсутствие плана. Командная работа специализирует роли (researcher, analyst, writer и т.д.) и даёт предсказуемый прогресс.

Как это выглядит на практике

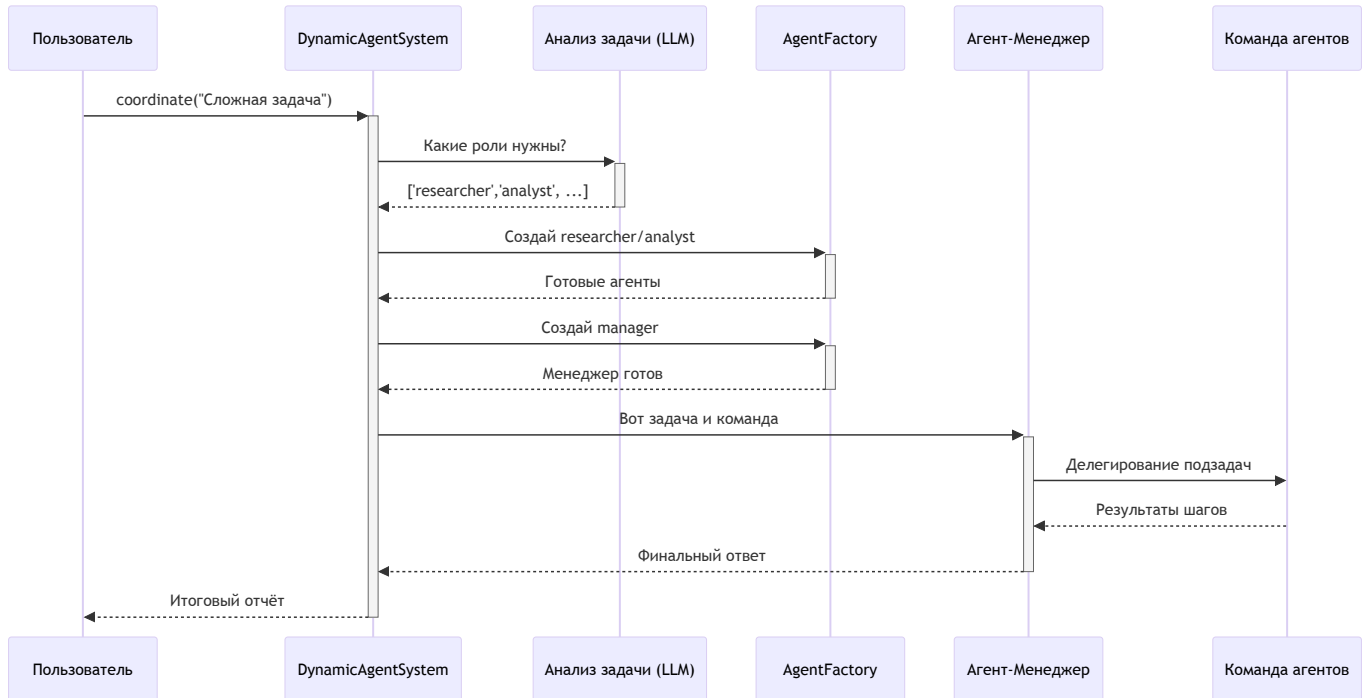
```
# main.py (упрощённый фрагмент)
import asyncio
from agent_system import DynamicAgentSystem

async def main():
    system = DynamicAgentSystem()
    task = "Проанализировать рынок электромобилей в 2024 году"
    report = await system.coordinate(task, show=True)
    print(report)

asyncio.run(main())
```

- `system.coordinate(...)` — точка входа; внутри: анализ задачи → подбор ролей → сборка команды → запуск менеджера → финальный отчёт.

Последовательность (высокоуровнево)



Ключевые шаги под капотом

1) Анализ задачи

```
# agent_system.py -> coordinate()
agent_types, pipeline_type = await self.analyze_task(initial_task)
```

LLM получает задачу + список доступных профилей и возвращает подходящие роли (например, researcher, analyst, manager).

2) Сборка команды

```
for agent_type in agent_types:
    if agent_type != 'manager':
        agent = self.factory.create_agent(agent_type, session_id, initial_task)
        self.agent_pool[agent.name] = {'agent': agent}
```

Фабрика по YAML-профилю собирает агента: модель, инструменты, политику памяти.

3) Назначение и запуск менеджера

```
manager = self.factory.create_agent('manager', session_id, ...)  
answer = manager.run(f"ЗАДАЧА ДЛЯ КООРДИНАЦИИ: {initial_task}")
```

Менеджер-агент декомпозирует, делегирует и синтезирует результат.

4) Формирование отчёта

```
report = [  
    "=== ИТОГОВЫЙ ОТЧЕТ ===\n",  
    f"🔍 Исходная задача: {initial_task}",  
    "\n 📄 Ответ менеджера:",  
    f"Подробный отчет:\n{answer}",  
]  
return "\n".join(report)
```

Итоги

- DynamicAgentSystem — единая точка входа для сложных задач.
- Динамически подбирает роли и собирает команду.
- Координирует выполнение через manager-агента.
- Возвращает собранный, читаемый итоговый отчёт.

Глава 3: Профили агентов (Agent Profiles)

Профиль агента — это его "должностная инструкция" в YAML. Он задаёт роль, инструменты, модель, политику памяти и системные инструкции. Благодаря профилям система гибкая: добавление нового агента — это просто новый YAML-файл без правок кода.

Зачем нужны профили

- Декларативность: описываем поведение, а не прошиваем его в код.

- Прозрачность: любой видит, чем агент занимается, какие у него инструменты и правила.
- Расширяемость: добавили `critic.yaml` — и система знает нового специалиста.

Анатомия профиля (пример)

```
# agent_profiles/researcher.yaml (упрощённый пример)

enable: true                # можно временно отключить профиль
model: model_search         # преднастроенная конфигурация LLM
type: code                  # code | tool_calling

# доступные инструменты
tools:
  - web_search
  - webpage_content

description: 'Эксперт по поиску и извлечению информации из интернета.'

# правила работы с памятью
memory_policy:
  scope_read: session       # видимость памяти: agent | session | global
  search_enabled: true      # разрешён семантический поиск
  last_k_steps: 5           # ограничение контекста шагов

# системные инструкции (суть роли)
prompt_templates: |-
  # Роль
  Ты — профессиональный исследователь. Твоя задача — находить актуальную
  и релевантную информацию по запросу.

  # Правила
  1. Всегда используй `web_search` для первичного поиска.
  2. Для чтения содержимого — `webpage_content`.
  3. Давай краткую сводку и ссылки на источники.
```

Полезные поля:

- `enable` : быстрый выключатель профиля.
- `model` : выбор пресета модели (например, `model_hard` , `model_lite` , `model_code`).
- `type` : базовый шасси агента (`code` для сложных задач, `tool_calling` — вызовы инструментов).

- `tools` : «навыки», передаваемые агенту.
- `description` : краткое резюме для выбора ролей в команде.
- `memory_policy` : как агент читает/пишет память (см. главу RAG-памяти).
- `prompt_templates` : ядро поведения агента.

Строгие контракты формата (пример из Text-to-SQL)

```
# agent_profiles/schema_rag_agent.yaml

enable: true
model: model_code
type: code

tools: ['schema_linking', 'get_distinct_values', 'schema_info']

description: 'Эксперт по связыванию схемы БД. Возвращает JSON.'
```

Это важно для конвейеров: следующий шаг ожидает строго определённый формат.

Загрузка профилей при старте

```
# agent_command.py (упрощённо)

def load_agent_profiles():
    profiles = {}
    for filename in os.listdir('agent_profiles'):
        if filename.endswith('.yaml'):
            agent_name = filename[:-5]
            data = yaml.safe_load(open(os.path.join('agent_profiles', filename), 'r'), e
            if data.get('enable', True):
                profiles[agent_name] = data
    return profiles

AGENT_PROFILES = load_agent_profiles()
```

AgentFactory просто берёт профиль из AGENT_PROFILES по имени.

Добавим нового агента за минуту (critic)

```
# agent_profiles/critic.yaml

enable: true
type: code
model: model_lite

description: "Оценивает текст и предлагает улучшения."

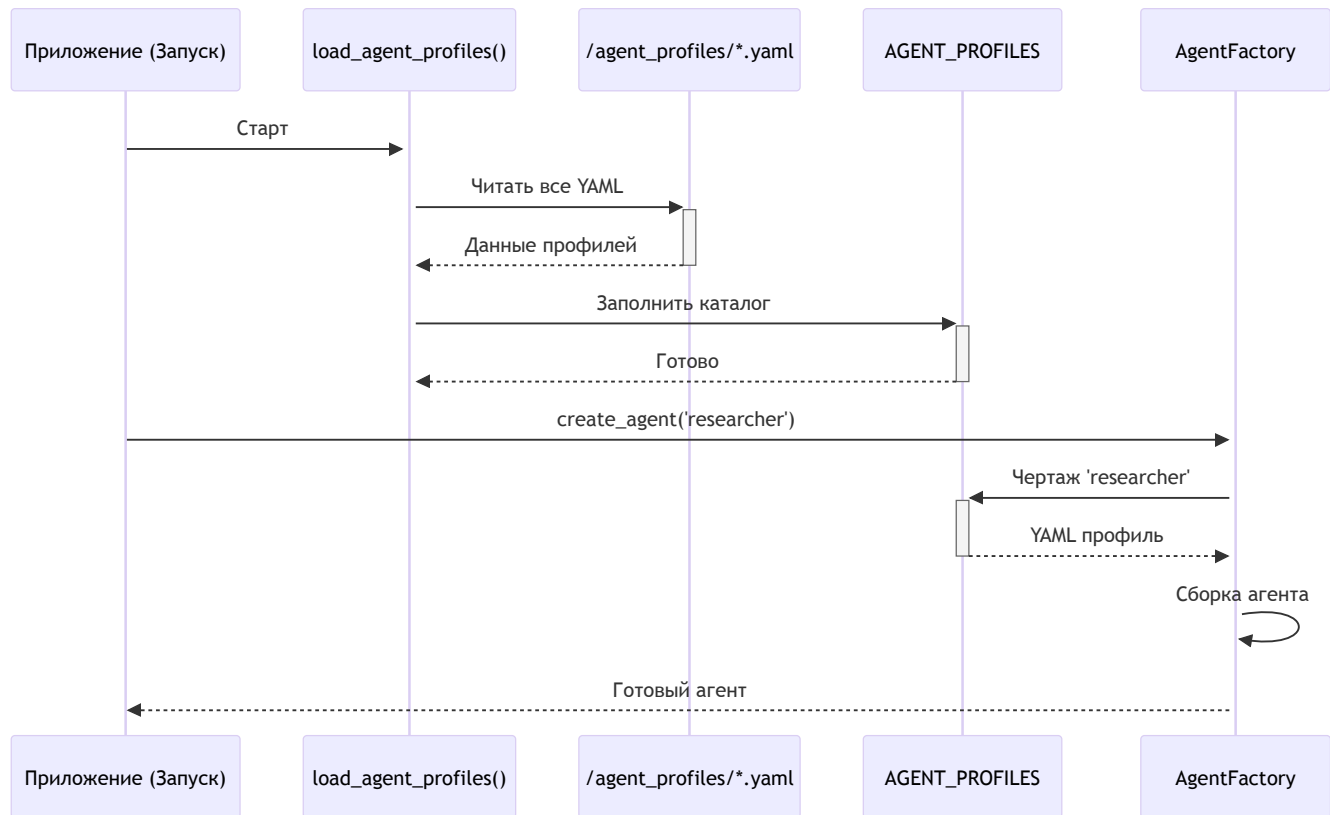
tools: []

memory_policy:
    provide_run_summary: false
    search_enabled: false

prompt_templates: |
    Ты — строгий, но справедливый критик.
    Выдели сильные стороны, укажи слабые и предложи конкретные улучшения.
```

Перезапуск — и агент доступен всей системе.

Визуализация процесса



Вывод

- Профили — фундамент гибкости системы.
- Чётко описывают роль, инструменты, модели и правила памяти.
- Лёгкое расширение без правок кода.

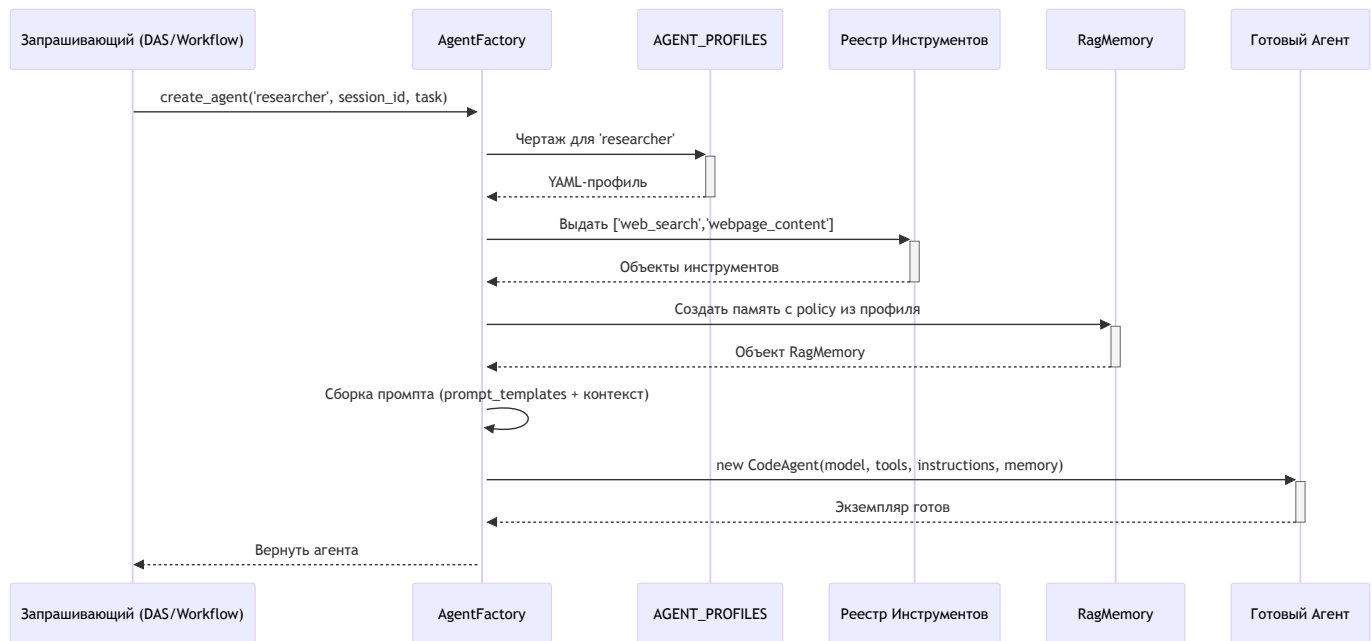
Глава 4: Фабрика агентов (AgentFactory)

AgentFactory — это сборочный цех и отдел кадров одновременно. По запросу (например, из DynamicAgentSystem) она берёт YAML-профиль, выдаёт инструменты, подключает память, прошивает инструкции и возвращает готового агента.

Задачи фабрики

1. Найти профиль в `AGENT_PROFILES` .
2. Создать/выдать инструменты по именам из профиля.
3. Подключить RAG-память с политикой из профиля.
4. Сформировать композитный промпт.
5. Создать экземпляр агента нужного типа (`code` / `tool_calling`).

Поток создания



Ключевые шаги в коде (упрощённо)

1) Профиль

```
if profile_type not in AGENT_PROFILES:
    raise ValueError(f"Неизвестный профиль: {profile_type}")
profile = AGENT_PROFILES[profile_type]
```

2) Инструменты

```
profile_tools = profile.get('tools', [])
tools = [self.tool_mapping.get(name) for name in profile_tools]
tools = [t for t in tools if t is not None]
```

`self.tool_mapping` заполняется при инициализации через `load_tools()` (см. главу про инструменты).

3) Память

```
rag_memory = create_rag_memory(
    session_id=session_id,
    agent_name=profile_type,
    profile_config=profile,
)
```

Используются поля `memory_policy` (например, `scope_read`, `search_enabled`, `strategic_write`).

4) Инструкции

```
def _build_composite_prompt(profile, session_id):
    base = profile.get('prompt_templates', '')
    extra = f"\nsession_id: {session_id}\n"
    return base + extra
```

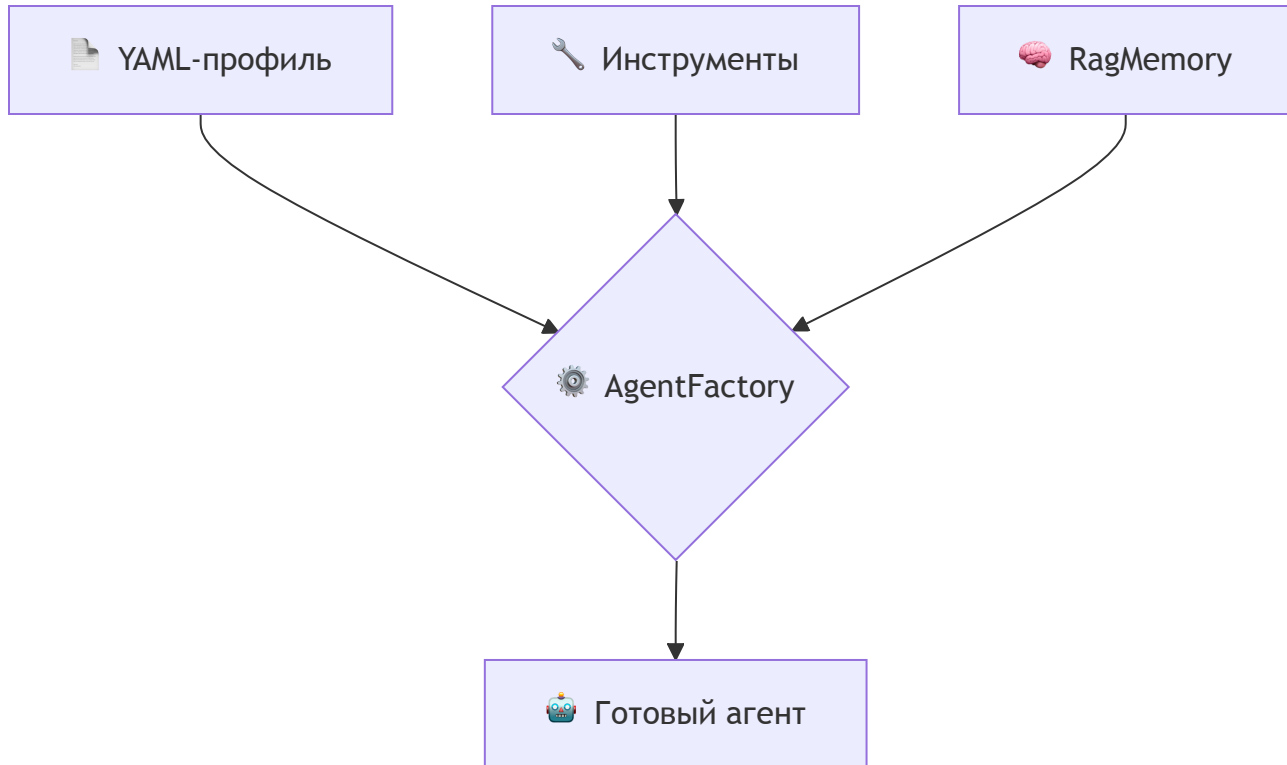
```
instructions = _build_composite_prompt(profile, session_id)
```

5) Экземпляр агента

```
agent_type = profile.get('type', 'code')
if agent_type == 'tool_calling':
    agent = ToolCallingAgent(model=profile.get('model'), tools=tools, instructions=inst
else:
    agent = CodeAgent(model=profile.get('model'), tools=tools, instructions=instruction

agent.memory = rag_memory
return agent
```

Визуализация конвейера



Примечания

- Инструменты могут поступать как из YAML-определений, так и из внешних источников (MCP), но фабрика работает с единым реестром.
- Профили отделяют конфигурацию от реализации. Любое изменение (модель, инструменты, политика памяти) — правкой YAML.
- Для потоков, требующих строгого формата вывода (например, Text-to-SQL), профиль может задавать жёсткий contract в `prompt_templates`.

Вывод

AgentFactory централизует сборку агентов и делает систему расширяемой: новый профиль — новый специалист без правок кода.

Глава 5: Система инструментов (Tool System)

Инструменты — это «руки» агентов: функции/классы для работы с файлами, вебом, БД, ML и т. д. Система инструментов централизует:

- декларативные описания (YAML) и динамическую загрузку;
- единый каталог `tool_mapping` для фабрики;
- телеметрию и логи через `ToolManager` .

Как устроено

- Описания лежат в `tool_definitions/*.yaml` .
- При старте выполняется `load_tools()` → формируется словарь имя → объект инструмента.
- `AgentFactory` при сборке читает `tools` из профиля и «выдаёт» готовые объекты.

Пример YAML-описания

```
# tool_definitions/duckduckgosearch.yaml
name: DuckDuckGoSearchTool
description: "Поиск информации в интернете"
category: "Веб"
source_type: class_instance
implementation_source: smolagents.DuckDuckGoSearchTool
parameters: []
```

Ключевые поля: `name` , `description` , `implementation_source` , `parameters` .

Загрузка инструментов (упрощённо)

```
def load_tools():
    tool_mapping = {}
    for filename in os.listdir('tool_definitions'):
        if filename.endswith('.yaml'):
            cfg = yaml.safe_load(open(os.path.join('tool_definitions', filename)))
            module_path, obj_name = cfg['implementation_source'].rsplit('.', 1)
            module = importlib.import_module(module_path)
            obj = getattr(module, obj_name)
            tool_mapping[cfg['name']] = obj() if cfg.get('source_type') == 'class_insta
# интеграция внешних (MCP) инструментов при наличии
return tool_mapping
```

Наблюдаемость: связка с ToolManager

- Вызовы инструментов оборачиваются `ToolManager.run_tool(...)`.
- Декоратор `@with_telemetry(name, description)` делает функцию «инструментом» с трассировкой и логами.

Добавление нового инструмента (пример)

1. Код:

```
# custom_tools/greeting.py
def say_hello() -> str:
    return "Hello, World!"
```

2. YAML:

```
# tool_definitions/say_hello.yaml
name: say_hello
source_type: custom_function
implementation_source: custom_tools.greeting.say_hello
```

3. Профиль агента:


```
# agent_profiles/researcher.yaml
tools:
  - DuckDuckGoSearchTool
  - say_hello
```

Перезапуск — и инструмент доступен всем агентам с этим профилем.

Вывод

- YAML-определения + динамическая загрузка дают расширяемость.
- Единый каталог инструментария упрощает выдачу навыков агентам.
- Интеграция с ToolManager обеспечивает телеметрию и контроль.

Глава 6: Определение инструмента (Tool Definition)

Определение инструмента — это YAML-«паспорт», описывающий имя, назначение и расположение кода. Такой подход позволяет добавлять новые навыки без правок ядра.

Схема YAML (пример)

```
# tool_definitions/file_write.yaml
name: file_write
description: "Записывает содержимое в файл. Для append=True — дописывает."
category: "Файлы"
source_type: custom_function
implementation_source: custom_tools.file_system_tools.file_write
parameters:
  - name: filename
    type: str
    description: Путь к файлу
    required: true
  - name: content
    type: str
    description: Содержимое для записи
    required: true
```

Ключевые поля: name , description , implementation_source , parameters .

Как YAML оживает

Запуском load_tools() все .yaml сканируются, код импортируется по implementation_source , и формируется словарь имя → функция/объект.

```
def load_tools():
    tool_mapping = {}
    for fn in os.listdir('tool_definitions'):
        if fn.endswith('.yaml'):
            cfg = yaml.safe_load(open(os.path.join('tool_definitions', fn)))
            module_path, obj_name = cfg['implementation_source'].rsplit('.', 1)
            module = importlib.import_module(module_path)
            obj = getattr(module, obj_name)
            tool_mapping[cfg['name']] = obj
    return tool_mapping
```

Выдача инструментов агентам

AgentFactory читает список `tools` из профиля и поднимает объекты из каталога `tool_mapping`, подключая их к агенту.

Вывод

YAML-определения делают инструменты самостоятельными и легко расширяемыми; динамическая загрузка убирает жёсткие зависимости из кода.

Глава 7: Менеджер инструментов (ToolManager)

ToolManager — диспетчер вызовов инструментов: добавляет логи, телеметрию и единообразный контроль исполнения.

Зачем нужен

- Централизованное логирование вызовов и ошибок.
- Трассировка (span) на каждый запуск инструмента.
- Простой способ «превратить» функцию в управляемый инструмент.

Декоратор @with_telemetry

```
from tool_manager import with_telemetry

@with_telemetry("image_analysis", "Анализ изображения")
def analyze_image_tool(image_path: str) -> str:
    # ... логика анализа ...
    return "{...json...}"
```

Каждый вызов будет логироваться и оборачиваться в телеметрию автоматически.

Центральный метод run_tool (упрощённо)

```
class ToolManager:
    def run_tool(self, tool_name: str, tool_function: Callable, task_description: str =
        span = None
    try:
        telemetry = get_telemetry_manager()
        span = telemetry.start_run_trace(run_id=session_id, agent_name=tool_name, t
        result = tool_function(**kwargs)
        return result
    except Exception as e:
        # запись ошибки в телеметрию/логи
        raise
    finally:
        if span:
            telemetry.finish_run_trace(span, success=True)
```

Контекстный менеджер tool_context

```
with get_tool_manager().tool_context("complex_tool", "Сложная задача", session_id) as c
    part = do_step()
    ctx.add_metadata("step", "done")
```

Дает тонкий контроль и возможность добавлять метаданные в ходе работы.

Где используется

- Агенты (через AgentFactory) вызывают инструменты, обёрнутые в телеметрию.
- UI/Streamlit-страницы могут запускать инструменты напрямую с логированием.

Вывод

ToolManager делает вызовы инструментов наблюдаемыми и предсказуемыми: меньше «чёрных ящиков», больше данных для диагностики и оптимизации.

Глава 8: Система памяти (RAG)

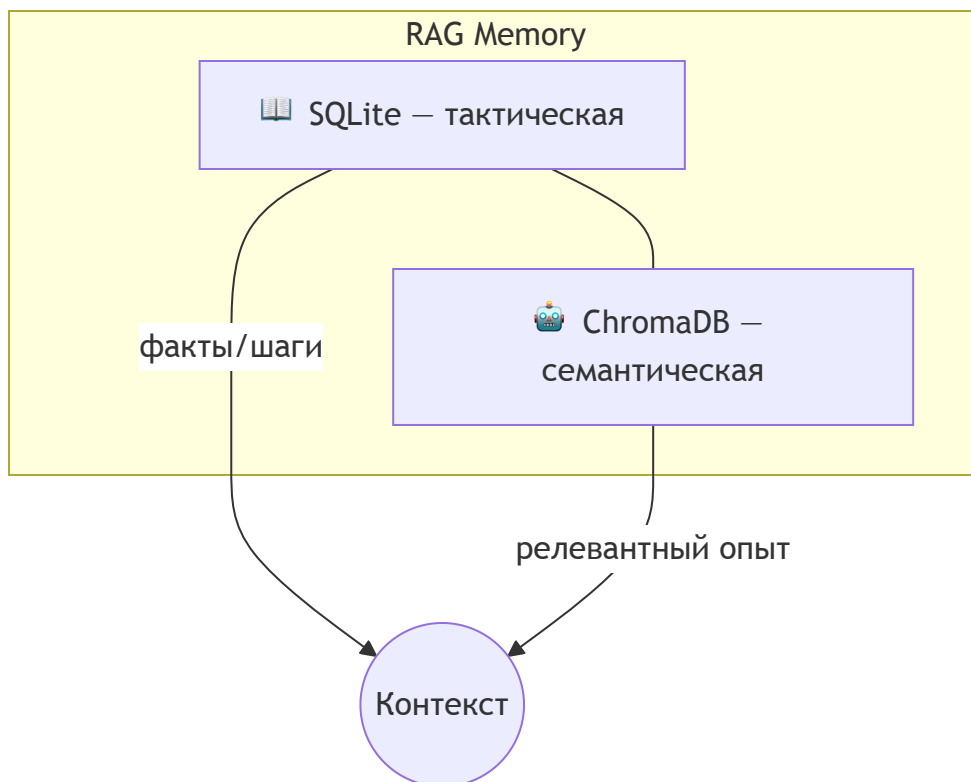
Внешний «мозг» агентов сочетает две части:

- Тактическая память (SQLite) — пошаговые факты и трассировка.
- Семантическая память (ChromaDB) — поиск по смыслу с эмбедами.

Зачем

- Учиться на опыте, не начинать каждую задачу с нуля.
- Делиться знаниями по правилам доступа.
- Экономить вызовы LLM и время.

Архитектура



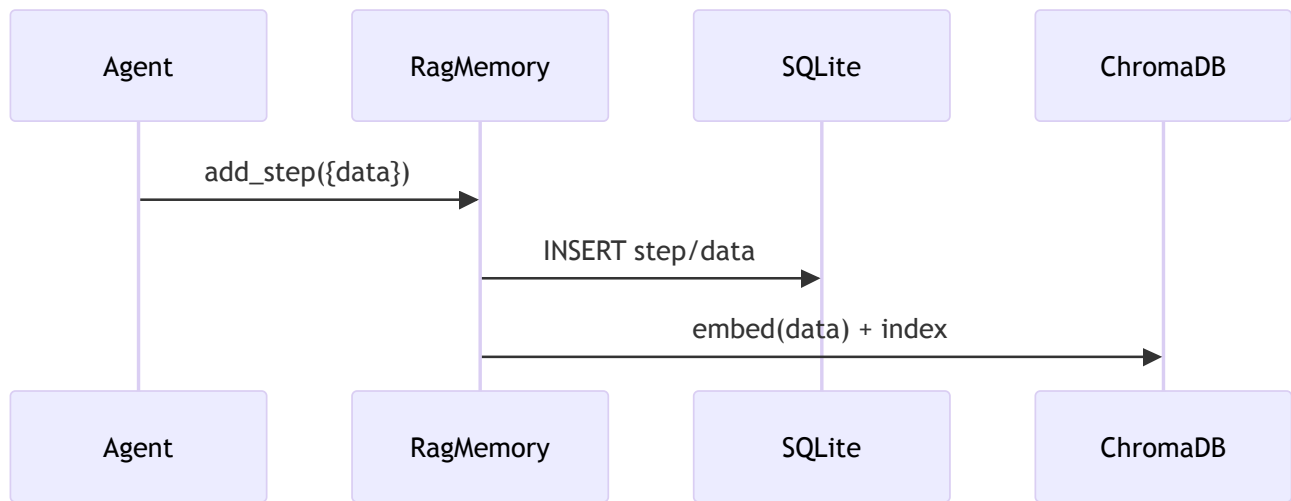
Политика памяти (memory_policy в профилях)

```
memory_policy:
  scope_read: session    # none | agent | session
  search_enabled: true
  last_k_steps: 5
  allow_add_step: true
  strategic_write: true  # для общих целей/контекста
```

- `scope_read` : область чтения (личная/сессия).
- `search_enabled` : включить семантический поиск.
- `last_k_steps` : сколько последних шагов включать.
- `allow_add_step` : разрешить запись шагов.
- `strategic_write` : писать стратегический контекст.

Жизненный цикл данных

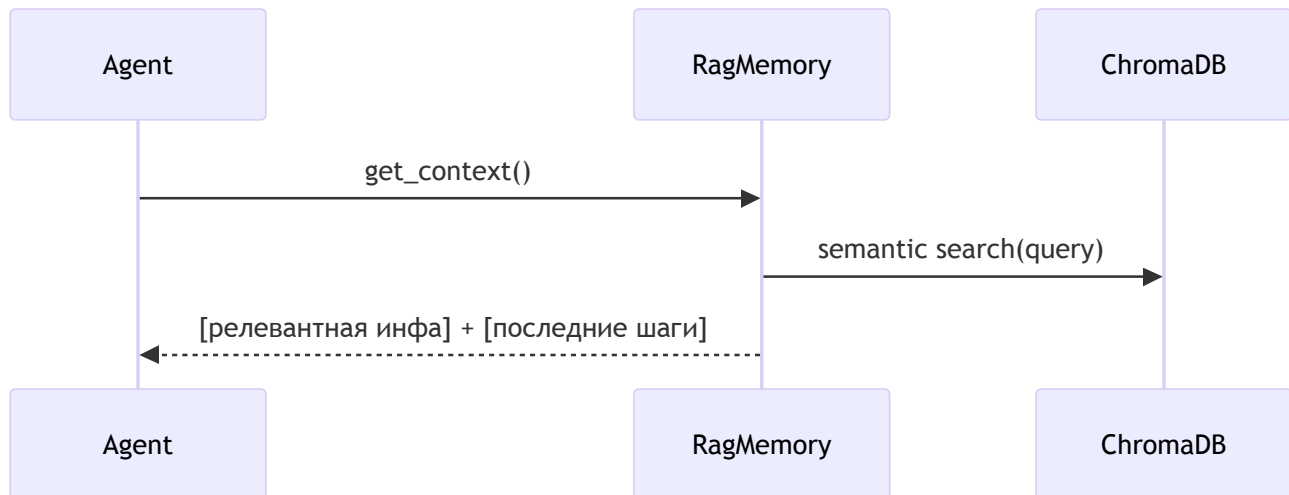
Сохранение опыта



Пример инструмента:

```
# memory/tools.py (упрощённо)
@tool
def save_memory(session_id: str, agent_name: str, data: Dict) -> int:
    # write to SQLite, add embedding to Chroma
    ...
```

Получение контекста



Пример API:

```
# memory/rag_memory.py (упрощённо)
class RagMemory:
    def get_context(self, max_tokens: int | None = None) -> str:
        parts = []
        if self.policy.search_enabled:
            parts.append(self._semantic_search(self.current_run_context))
        parts.append(self._last_steps(self.policy.last_k_steps))
        return "\n\n".join(p for p in parts if p)
```

Инициализация БД

```
# memory/database.py (фрагмент)
class DatabaseHandler:
    def _init_chroma(self, embedding_model: str):
        self.embedding_model = SentenceTransformer(embedding_model)
        self.chroma_client = chromadb.PersistentClient(path=self.chroma_path)
        self.tactical_collection = self.chroma_client.get_or_create_collection("tactical")
```

Вывод

- Связка SQLite+ChromaDB превращает память в обучаемую библиотеку.
- Политики управляются из YAML-профилей, без правок кода.
- `save_memory` / `get_context` закрывают большую часть сценариев.

Глава 9: Менеджер памяти (MemoryManager)

Централизованный «библиотекарь» памяти: единая точка входа к SQLite/ChromaDB, потокобезопасность и интеллектуальное разрешение конфликтов.

Зачем

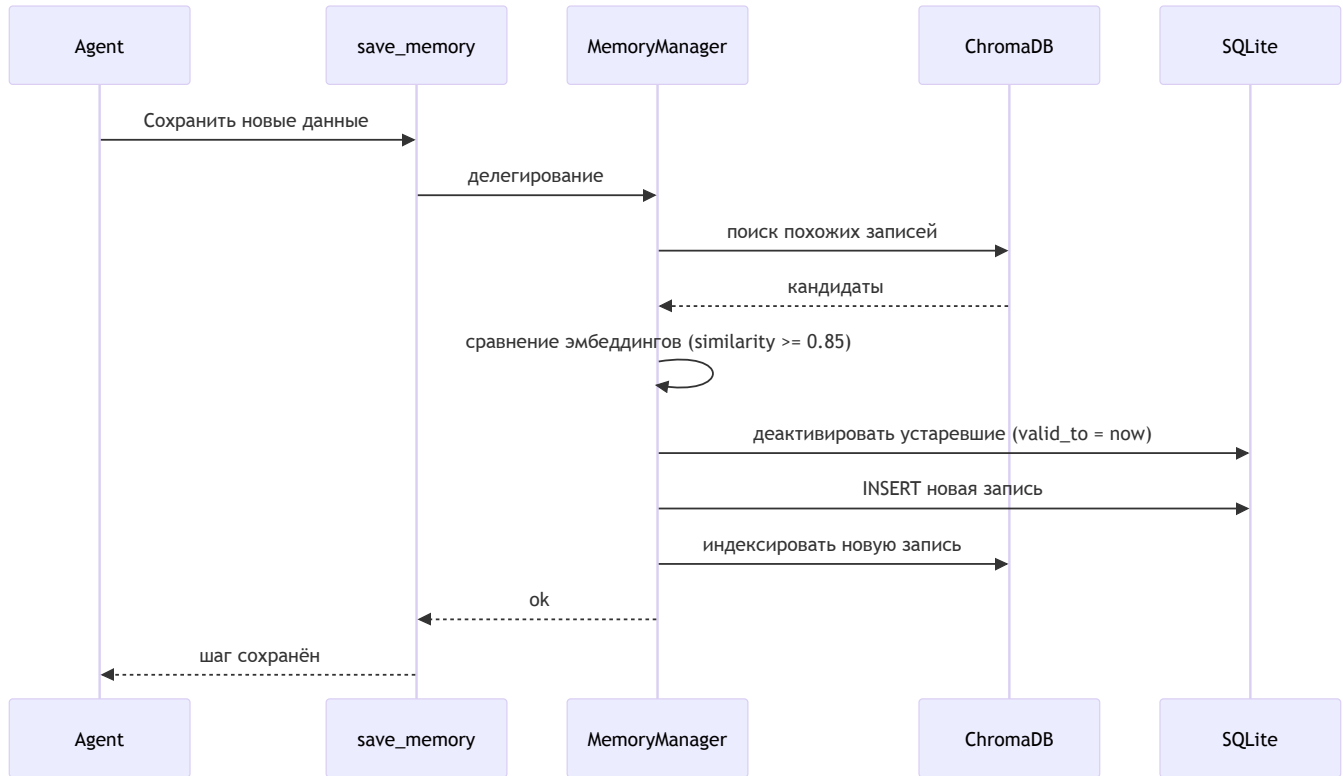
- Исключить гонки и дубли при одновременной записи.
- Скрыть детали БД от агентов и инструментов.
- Поддерживать актуальность знаний (деактивация устаревших записей).

Роль и доступ

```
# memory/manager.py (упрощённо)
from .manager import get_memory_manager
memory_manager = get_memory_manager() # синглтон
```

Все операции с памятью (через инструменты `save_memory` , `get_memory`) проксируются в менеджер.

Поток сохранения с разрешением конфликтов



Ключевые фрагменты логики

Поиск и выявление конфликтов

```
def _resolve_conflicts(self, session_id: str, agent_name: str, new_data: Dict) -> list:
    new_text = self._extract_text_content(new_data)
    if not new_text:
        return []
    where = {"$and": [{"session_id": {"$eq": session_id}}, {"agent_name": {"$eq": agent_name}}]
    candidates = self._search_semantic(self.db_handler.tactical_collection, new_text, n_results=5)
    new_emb = self._create_embedding(new_text)
    conflicts = []
    for tid in candidates:
        existing_emb = self._get_embedding(tid)
        if cosine_sim(new_emb, existing_emb) >= 0.85:
            conflicts.append(tid)
    return conflicts
```

Деактивация устаревших записей

```
def _deactivate_conflicting_records(self, conflicts: list):
    if not conflicts:
        return
    now = datetime.now().isoformat()
    for (session_id, agent_name, step) in conflicts:
        cursor.execute("UPDATE agent_memory SET valid_to = ? WHERE session_id=? AND age
    conn.commit()
```

Потокобезопасность и единый доступ

- Синглтон + последовательная обработка запросов устраняют гонки за ресурсы.
- Инкапсуляция подключения/курсов в DatabaseHandler .

API для UI

- MemoryRAGManager (streamlit API): статус, поиск, перестроение индекса.

Вывод

MemoryManager превращает множество личных дневников (RagMemory) в единый, целостный и актуальный корпоративный архив, сохраняя консистентность и удобство работы для агентов.

Глава 10: Определение Workflow (WorkflowDefinition)

WorkflowDefinition — декларативный «чертёж» процесса в YAML: шаги, зависимости, условия и политики. Движок читает его и исполняет без хаоса.

Зачем

- Прозрачный план: человек и машина понимают один и тот же YAML.

- Надёжность: строгие зависимости исключают гонки и пропуски.
- Гибкость: правки без изменений кода.

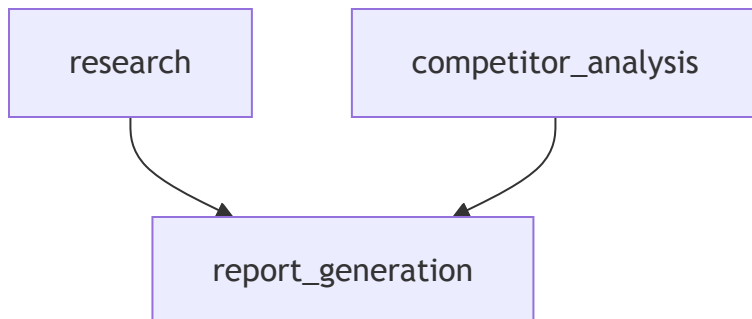
Пример YAML

```
# workflow_pipelines/ev_market_report.yaml
name: "EV Market Report"
description: "Сбор, анализ, отчёт по рынку электромобилей"
inputs:
  topic: "рынок электромобилей"
  competitors: ["Tesla", "BYD", "Volkswagen"]
steps:
  - id: research
    agent_type: researcher
    task: "Собери материалы по теме '{topic}' за последний год"

  - id: competitor_analysis
    agent_type: analyst
    task: "Проанализируй позиции: {competitors}"
    depends_on: [research]

  - id: report_generation
    agent_type: writer
    task: "Собери итоговый отчёт из research и competitor_analysis"
    depends_on: [research, competitor_analysis]
```

Граф зависимостей:



Расширения

- condition: условное выполнение шага.

- `retry_policy`: `max_retries`, стратегия бэкоффа.
- `parallel_execution`: независимые шаги — параллельно.
- `step_type`: `tool` — прямой вызов инструмента без агента.

От YAML к объектам Python

```
# workflow/models.py (идея)
@dataclass
class WorkflowStep:
    id: str
    task: str
    agent_type: str | None = None
    depends_on: list[str] = field(default_factory=list)
    retry_policy: RetryPolicy | None = None

@dataclass
class WorkflowDefinition:
    name: str
    description: str
    steps: list[WorkflowStep]

    @classmethod
    def from_yaml(cls, yaml_path: str) -> "WorkflowDefinition":
        data = yaml.safe_load(open(yaml_path, 'r', encoding='utf-8'))
        return cls.from_dict(data)
```

Вывод

`WorkflowDefinition` переносит сложность из кода в декларативную структуру: проще править, легче контролировать и повторно использовать.

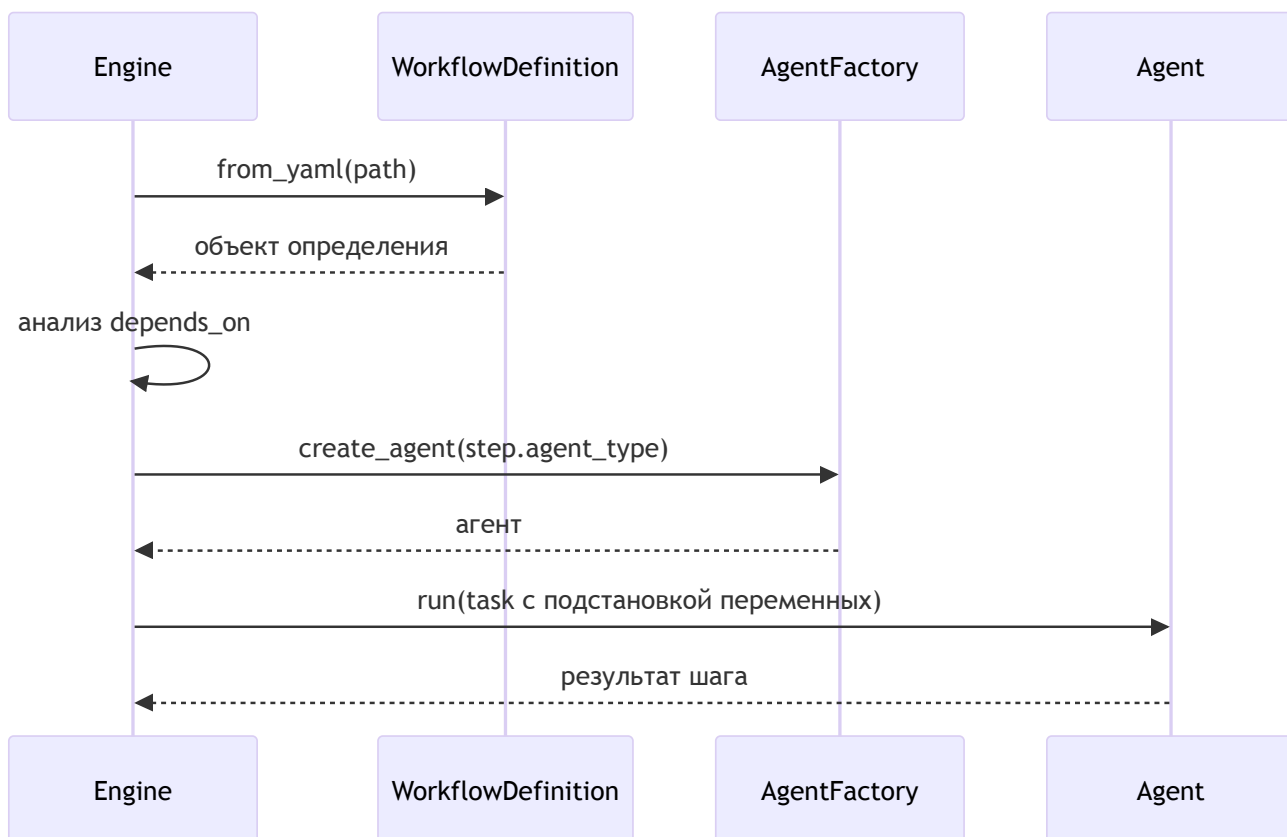
Глава 11: Движок рабочих процессов (WorkflowEngine)

WorkflowEngine — «прораб», который читает чертёж WorkflowDefinition и исполняет шаги в правильном порядке, создавая нужных агентов и передавая данные между шагами.

Как используется

```
engine = WorkflowEngine()
result = await engine.execute_workflow_from_yaml(
    yaml_path="workflow_pipelines/ev_market_report.yaml",
    topic="рынок беспилотных авто",
    competitors=["Waymo", "Cruise", "Zoox"],
)
```

Поток выполнения



Ключевые методы (упрощённо)

Загрузка и запуск:

```
async def execute_workflow_from_yaml(self, yaml_path: str, **vars):
    workflow_def = WorkflowDefinition.from_yaml(yaml_path)
    context = WorkflowContext(variables=vars)
    return await self.execute_workflow(workflow_def, context)
```

Последовательное исполнение и зависимости:

```
async def _execute_steps_sequential(self, workflow_def, context):
    step_results = {}
    for step in workflow_def.steps:
        if not self._check_step_dependencies(step, step_results):
            continue
        step_results[step.id] = await self._execute_workflow_step(step, context, workflow_def)
        if step_results[step.id].status != StepStatus.COMPLETED:
            raise WorkflowExecutionError(f"Шаг {step.id} провален")
    return step_results
```

Исполнение шага с повторами:

```
async def _execute_workflow_step(self, step, context, workflow_def):
    async def step_executor(exec_ctx):
        formatted = self._format_task_with_variables(step.task, context, step.id)
        return await self._execute_agent_step(step, context, formatted)
    return await self.retry_engine.execute_with_retry(step_id=step.id, step_func=step_executor)
```

Вызов агента:

```
async def _execute_agent_step(self, step, context, task: str):
    agent = self.factory.create_agent(step.agent_type, context.session_id, task)
    return agent.run(task, stream=False)
```

Шаги-инструменты

```
- id: generate_visual
  step_type: tool
  tool_name: generate_image_tool
  tool_params:
    prompt: "Иллюстрация: {topic}"
    session_id: "{session_id}"
```

Такие шаги выполняются быстрее и дешевле — без LLM-агента.

Надёжность: RetryPolicy

```
- id: research
  agent_type: researcher
  task: "Ищи: {topic}"
  retry_policy:
    max_retries: 3
    backoff_strategy: exponential
```

Движок прозрачно повторит шаг при временных сбоях.

Вывод

WorkflowEngine выполняет YAML-планы предсказуемо и устойчиво: соблюдает зависимости, подставляет переменные, поддерживает шаги-инструменты и автоматические повторы.

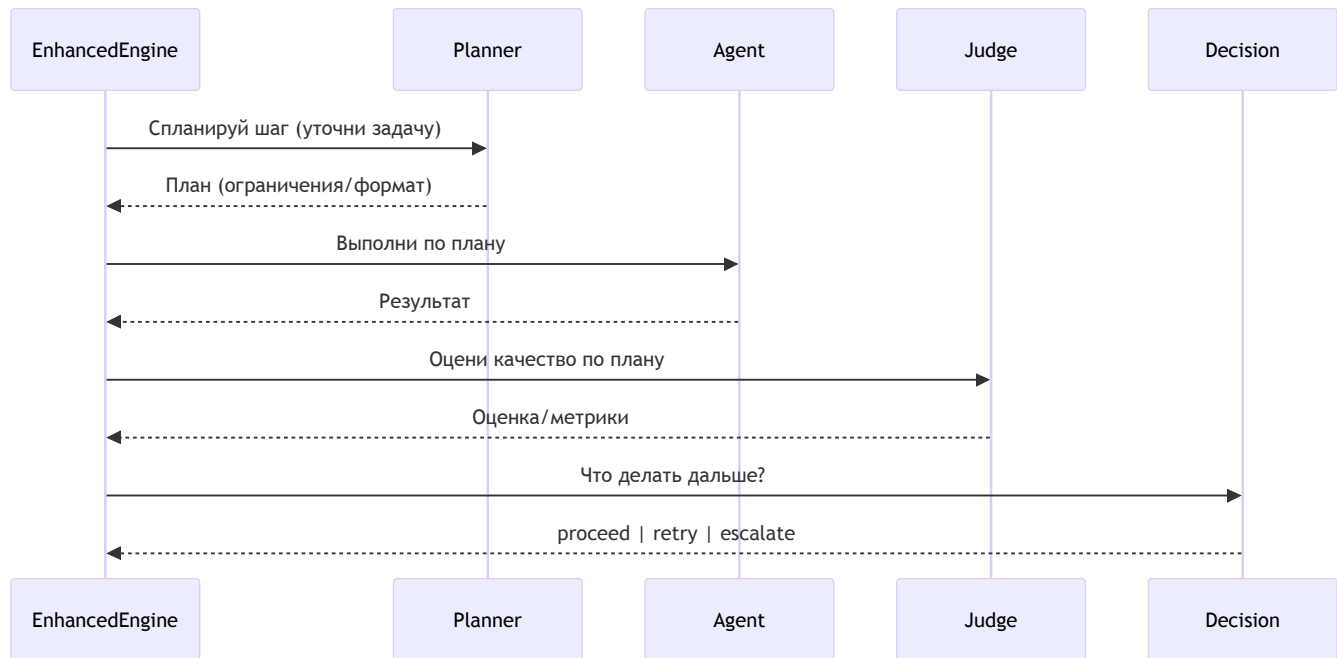
Глава 12: Расширенный движок процессов (EnhancedWorkflowEngine)

EnhancedWorkflowEngine добавляет поверх WorkflowEngine интеллектуальный цикл качества и устойчивости: планирование шага, оценку результата, решение о продолжении/повторе и защиту от сбоев.

Зачем

- Качество: проверять, «насколько хорошо» выполнен шаг, а не только «выполнен ли».
- Самокоррекция: при низком качестве — доуточнить задачу и повторить.
- Устойчивость: выключать «падающих» исполнителей и экономить ресурсы.

Цикл выполнения



Ключевые точки (упрощённо)

Одна попытка шага:

```
async def _execute_single_step_attempt(self, step, wf_ctx):
    plan = await self.planner.plan_step(step, wf_ctx)
    result = await self.circuit_breaker_manager.call_agent_safely(
        agent_name=step.agent_type,
        agent_func=self._execute_step_with_policy,
    )
    verdict = await self.judge.validate_result(result, plan)
    decision = await self.decision_engine.make_decision(verdict)
    if decision.action != "proceed":
        raise Exception(f"{decision.action}: {decision.reason}")
    return result
```


Адаптивные повторы:

```
return await self.retry_engine.execute_with_retry(  
    step_id=step.id,  
    step_func=self._execute_single_step_attempt,  
    on_retry_modify_context_func=self._apply_decision_modifications,  
)
```

Повторная попытка может уточнить задачу/контекст по рекомендации `decision_engine`.

Circuit Breaker:

```
result = await self.circuit_breaker_manager.call_agent_safely(  
    agent_name=step.agent_type,  
    agent_func=...  
)
```

При серии сбоев «выключатель» размыкается и предотвращает дальнейшие вызовы проблемного агента.

Конфигурация

```
# workflow/config/enhanced_global.yaml  
features:  
  pre_step_planner: { enabled: true }  
  post_step_judge:  { enabled: true }  
  circuit_breaker:  { enabled: true }  
  
# workflow/config/policies/default.yaml  
quality_gates:  
  default:  
    min_quality_score: 0.7  
    hard_fail_threshold: 0.3
```

Вывод

EnhancedWorkflowEngine добавляет «разум» к конвейеру: планирует, проверяет, принимает решения и защищает процесс, обеспечивая предсказуемое качество и устойчивость.

Глава 13: Событийно-управляемый Workflow Engine

Расширение WorkflowEngine для реактивной автоматизации: рабочие процессы запускаются по событиям, по времени или по условиям.

Понятия

- Событие (Event): факт, что-то произошло (например, `workflow.completed` , `new_file.uploaded`).
- Триггер (Trigger): правило «если событие/время/условие, то запустить workflow X».

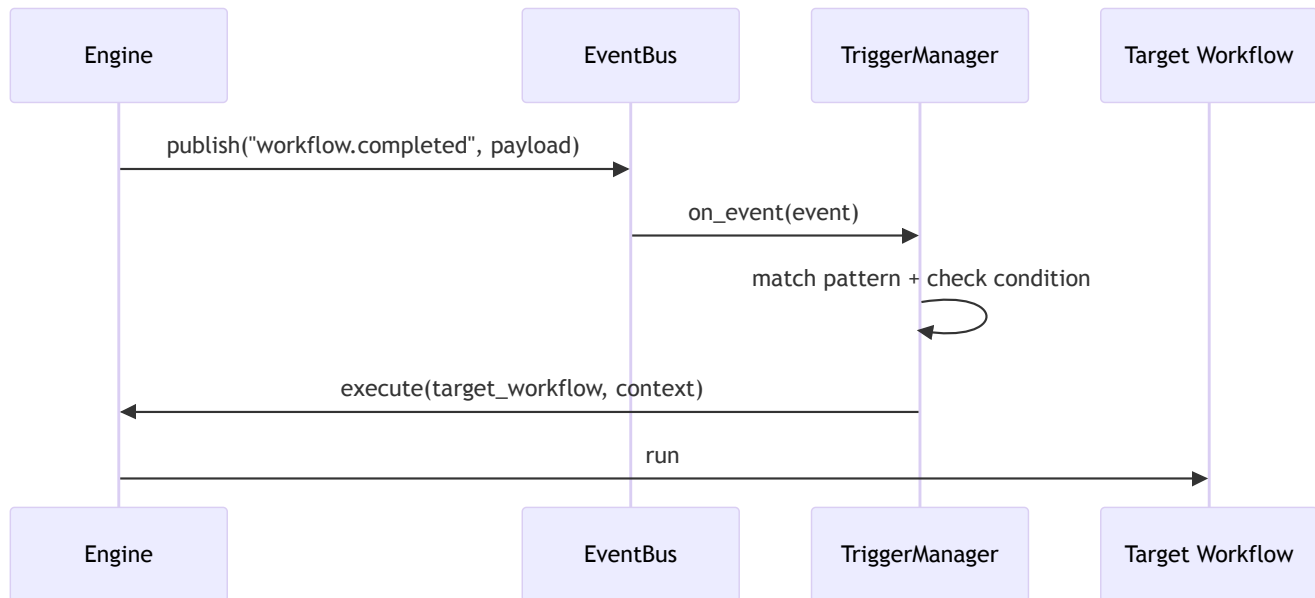
Типы триггеров

- EVENT_BASED: реагирует на событие по шаблону (`workflow.completed`).
- TIME_BASED: cron/расписание (ежедневные задачи).
- CONDITION_BASED: периодическая проверка условия.

Пример конфигурации

```
# config/triggers.yaml
triggers:
  report_on_sql_success:
    enabled: true
    trigger_type: event_based
    event_pattern: "workflow.completed"
    condition: "event.payload.workflow_name == 'Text-to-SQL Pipeline'"
    target_workflow: "generate_and_email_report"
```

Архитектура



Обогащение контекста (ContextEnrichment)

Перед запуском целевого workflow триггер может подтянуть данные из памяти (RAG) и внедрить их как входные переменные.

```
triggers:
  report_on_sql_success:
    target_workflow: "generate_and_email_report"
    context_enrichment:
      memory_queries:
        - query: "Последний сгенерированный SQL-запрос"
          inject_as: "last_sql_query"
          session_id: "{event.payload.client_id}"
```

Вывод

Event-Driven Workflow Engine добавляет реактивность: процессы соединяются через события и легко настраиваются YAML-правилами, а контекст можно обогащать из памяти для более точной автоматизации.

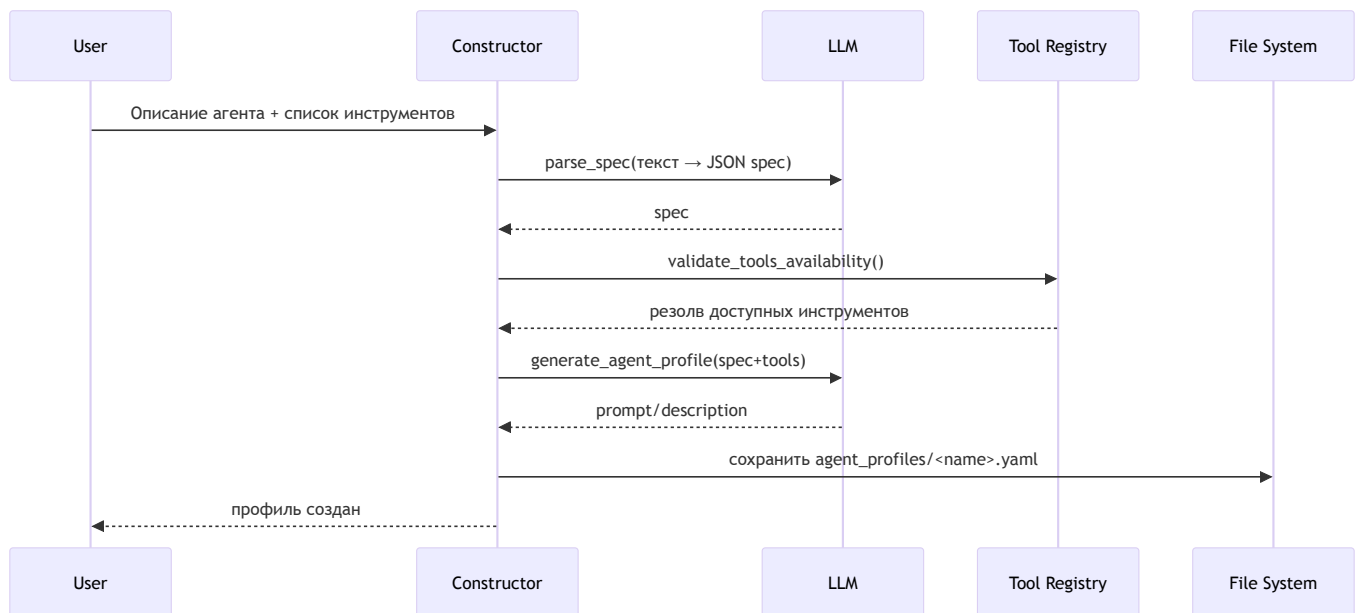
Глава 14: Конструктор агентов (Agent Constructor)

Мета-агент, который по текстовому описанию генерирует готовый YAML-профиль нового агента: быстро, последовательно и без ошибок в синтаксисе.

Что делает

- Анализирует описание роли и задач (LLM).
- Проверяет доступность запрошенных инструментов.
- Планирует минимальные зависимости/порядок.
- Генерирует YAML-профиль с description/prompt/tools/model.

Конвейер из 4 шагов



Ключевые операции (упрощённо)

Анализ спецификации:

```
def parse_spec(description: str) -> dict:
    raw = call_openai_api(prompt=description, system_prompt="Верни ТОЛЬКО JSON по схеме")
    return normalize(raw)
```

Проверка инструментов:

```
def validate_tools_availability(tools: list[str]) -> dict:
    custom = set(_load_custom_tool_names())
    mcp = set(_load_mcp_tool_names())
    resolved, missing = [], []
    for t in tools:
        if t in custom: resolved.append({"type": "custom", "name": t})
        elif t in mcp: resolved.append({"type": "mcp", "name": t})
        else: missing.append(t)
    return {"tools_resolved": resolved, "unavailable_tools": missing}
```

Генерация профиля:

```
def generate_agent_profile(spec: dict, tools: list[dict]) -> str:
    prompt = call_openai_api(...)
    desc = call_openai_api(...)
    profile = {
        "type": "tool_calling",
        "description": desc,
        "model": "model_code",
        "tools": [t["name"] for t in tools],
        "prompt_templates": prompt,
        "enable": True,
    }
    path = f"agent_profiles/{spec['agent_name']}.yaml"
    save_yaml(path, profile)
    return path
```

Пример результата

```
# agent_profiles/financial_analyst.yaml
type: tool_calling
model: model_code
description: "Агент-аналитик по CSV. Находит строки с максимальной прибылью."
tools:
  - file_read
  - csv_parser
prompt_templates: |
  Ты — точный и педантичный финансовый аналитик...
  1) Прочитай файл через file_read
  2) Разбери csv_parser
  3) Найди максимальную прибыль
  4) Верни структурированный вывод
enable: true
```

Примечание

Сам Конструктор — тоже агент с собственным профилем и наборами инструментов, вызываемый как обычный агент внутри системы.

Глава 15: Инструменты Конструктора Агентов

Набор инструментов для метагента `agent_constructor`, который создаёт новых агентов «под задачу».

Последовательность

1. `spec_parser` — нормализует текст в JSON-спецификацию.
2. `tools_availability_validator` — проверяет наличие запрошенных инструментов (custom/MCP).
3. `dependency_planner` — формирует простой план/порядок (задел для расширения).
4. `agent_profile_generator` — генерирует `description/prompt` и собирает YAML-профиль.

1. spec_parser

```
def parse_spec(description: str) -> dict:
    sys = "Верни ТОЛЬКО JSON по схеме {...}"
    raw = call_openai_api(prompt=description, system_prompt=sys)
    return normalize(raw)
```

Результат: {agent_name, persona, goals, inputs, outputs, ...}

2. tools_availability_validator

```
def validate_tools_availability(tools: list[str]) -> dict:
    custom = set(_load_custom_tool_names())
    mcp = set(_load_mcp_tool_names())
    resolved, missing = [], []
    for t in tools:
        if t in custom: resolved.append({"type": "custom", "name": t})
        elif t in mcp: resolved.append({"type": "mcp", "name": t})
        else: missing.append({"tool": t, "reason": "NOT_FOUND"})
    return {"tools_resolved": resolved, "unavailable_tools": missing}
```

Гарантирует, что профиль не будет ссылаться на несуществующие инструменты.

3. dependency_planner (упрощённый)

```
def plan_dependencies(tools_resolved: list[dict]) -> dict:
    nodes = [t["name"] for t in tools_resolved]
    return {"tools_ordered": nodes, "graph": {"nodes": nodes, "edges": []}}
```

Подготовка для будущих сложных сценариев.

4. agent_profile_generator

```
def generate_agent_profile(spec: dict, tools_resolved: list[dict]) -> str:
    prompt = call_openai_api(...)
    desc = call_openai_api(...)
    profile = {
        "type": "tool_calling",
        "description": desc,
        "model": "model_code",
        "tools": [t["name"] for t in tools_resolved],
        "prompt_templates": prompt,
        "enable": True,
    }
    path = f"agent_profiles/{spec['agent_name']}.yaml"
    save_yaml(path, profile)
    return path
```

Формирует финальный YAML-файл и сохраняет его в `agent_profiles/` .

Глава 16: Оптимизатор промптов (PromptOptimizer)

Автоматически улучшает системные инструкции агентов под конкретную модель (семейство/ версию), повышая качество и стабильность ответов.

Зачем

- Снимает рутину промпт-инжиниринга и адаптации под разные LLM.
- Делает инструкции структурированными и воспроизводимыми.
- Хранит метаданные оптимизаций и бэкапит профили.

Режимы запуска

- Автоматически при старте UI (`optimize_all_agents()`).
- Из CLI с предпросмотром:


```
python prompt_optimizer/optimize_agents.py --list
python prompt_optimizer/optimize_agents.py --agents researcher --dry-run
python prompt_optimizer/optimize_agents.py --agents researcher
```

Как работает

Идея: «ИИ улучшает инструкции для ИИ». Оптимизатор строит мета-промпт с контекстом (модель, инструменты, лучшие практики) и просит модель переписать базовый промпт.

Итерация по агентам и защита от повторной оптимизации:

```
def optimize_all_agents(self):
    for agent_name in agents_to_optimize:
        profile = AGENT_PROFILES[agent_name]
        model_info = self.get_model_info(profile.get('model'))
        if profile.get('optimization_metadata', {}).get('optimizer_model') == model_info:
            continue
        # ... выполнить optimize_prompt и update_profile ...
```

Мета-промпт (идея):

```
## Task
Re-write Baseline Prompt into Revised Prompt using Best Practices.
CRITICAL: You optimize a prompt FOR YOURSELF (your model family/version).
## Agent Context
- Agent Name: <name>
- Model: <family/version>
- Tools: <list>
## Best Practices
- структура, формат вывода, примеры, валидация...
## Baseline Prompt
<original_prompt>
```

Сохранение и бэкап:

```
def update_profile(...):
    backup(original_yaml)
    profile['prompt_templates'] = revised_prompt
    profile['optimization_metadata'] = {
        'optimizer_model': model_info['model_name'],
        'optimized_at': now_iso,
    }
    save_yaml(profile_path, profile)
```

Практика

- Используйте --dry-run для сравнения «до/после» без записи.
- Запускайте повторно при смене модели — оптимизатор адаптирует промпт под новое семейство.

Вывод

PromptOptimizer автоматизирует улучшение инструкций: быстрее, качественнее и с учётом конкретной LLM, сохраняя историю изменений.

Глава 17: Пайплайн Text-to-SQL

Конвейер, превращающий вопрос на естественном языке в безопасный SQL и итоговый ответ, используя Workflow + инструменты + агентов.

Зачем

- Отделяет этапы NLU, связывания со схемой, генерации/проверки SQL и выполнения.
- Делает процесс прозрачным, расширяемым и безопасным.

Этапы

1. NLU/Intent: разбор запроса, извлечение сущностей/метрик/фильтров.
2. Schema Linking: сопоставление сущностей с таблицами/колонками, JOIN-контекст.

3. SQL Generation/Verify/Execute: генерация SQL → проверка на безопасность → выполнение и форматирование результата.

YAML-чертёж (фрагменты)

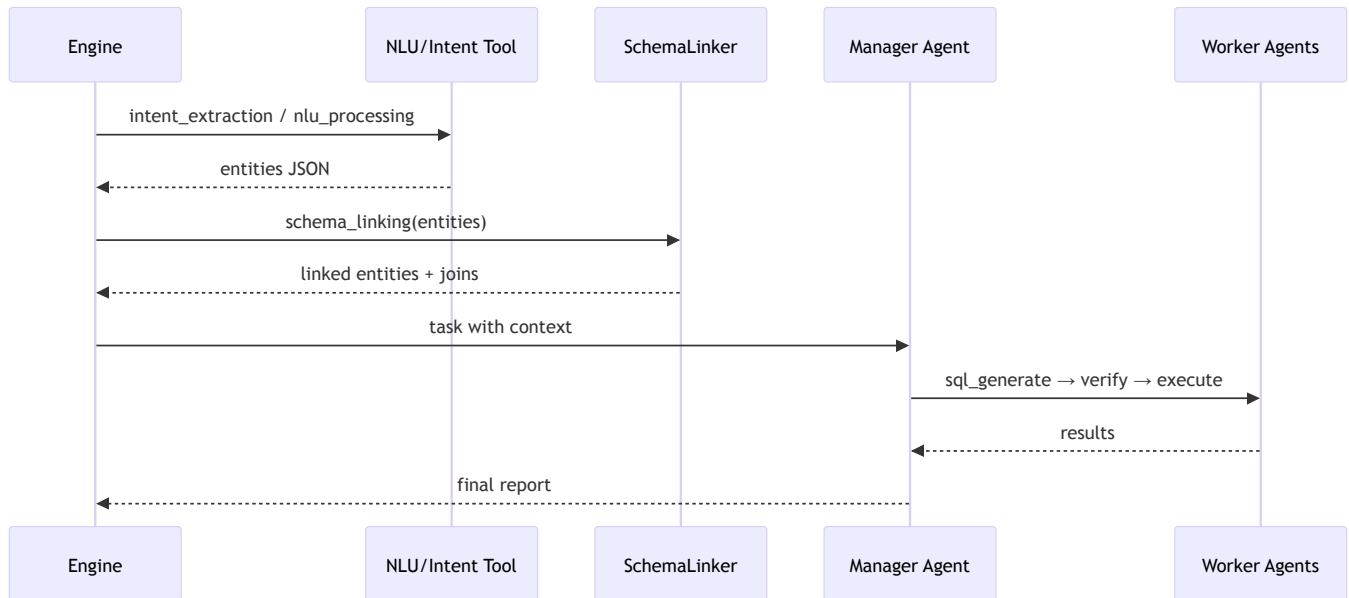
Шаги-инструменты (быстро и дешево):

- id: nlu_processing
step_type: tool
tool_name: natural_language_processing
tool_params:
text: '{query}'
- id: intent_extraction_step
step_type: tool
tool_name: intent_extraction
tool_params:
text: '{query}'
- id: schema_linking_step
step_type: tool
tool_name: schema_linking
tool_params:
entities: '{intent_extraction_step}'
depends_on: [nlu_processing, intent_extraction_step]

Шаг-агент (мини-оркестрация):

- id: sql_pipeline
step_type: agent
agent_type: manager
task: |
Координируй генерацию, проверку и выполнение SQL.
Контекст: {query}, {nlu_processing}, {intent_extraction_step}, {schema_linking_step}
1) Вызови sql_generator_agent
2) Вызови sql_verifier_agent
3) Вызови db_audit_agent
depends_on: [schema_linking_step]

Поток выполнения



Безопасность

- Шаг проверки (verifier) блокирует опасные конструкции (DROP/DELETE и т. п.).
- DB audit/обёртки на уровне драйвера ограничивают права и шорт-листяют операции.
- Политики повторов для сетевых/LLM-сбоев.

Кодовые опоры (идеи)

Инструменты:

```
# custom_tools/text_to_sql/core.py
nlu_processor = NLUProcessor()

def intent_extraction(text: str, session_id: str=None) -> dict:
    return nlu_processor.extract_intent(text)
```

Генератор SQL:

```
# custom_tools/text_to_sql/sql_generator.py
class SQLGenerator:
    def generate_sql(self, context: str, user_query: str) -> dict:
        for attempt in range(self.max_retries):
            prompt = build_sql_generation_prompt(context, user_query)
            resp = call_openai_api(prompt=prompt)
            # validate / normalize
            if is_valid:
                return {"sql_query": sql_query}
        return {"error": "failed"}
```

Вывод

Пайплайн Text-to-SQL — модульная и безопасная архитектура: шаги-инструменты для NLU/связывания, агент-менеджер для генерации/проверки/выполнения SQL, управляемые YAML-описанием процесса.

Глава 18: Система плагинов для БД (DBPluginManager)

Универсальный адаптер для разных СУБД (SQLite/Postgres/...): единый интерфейс, независимый от диалектов SQL и драйверов.

Зачем

- Единообразный доступ к БД из агентов/пайплайнов.
- Расширяемость через плагины (по схеме DSN).
- Интроспекция схем, тест соединения, безопасное выполнение запросов.

Как устроено

Реестр плагинов:

```
# db_plugins/manager.py
_PLUGINS = {
    "sqlite": SQLitePlugin(),
    "postgres": PostgresPlugin(),
    "postgresql": PostgresPlugin(),
    "duckdb": DuckDBPlugin(),
}
```

Выбор плагина по DSN:

```
from urllib.parse import urlparse

def get_plugin(dsn: str):
    scheme = urlparse(dsn).scheme.lower()
    plugin = _PLUGINS.get(scheme)
    if not plugin:
        raise ValueError(f"Нет плагина для схемы: {scheme}")
    return plugin
```

Интерфейс плагина (идея):

- connect(dsn)
- introspect_schema(conn)
- execute_select(conn, sql, ...)
- explain(conn, sql)

API для UI

```
# db_plugins/streamlit_api.py
mgr = get_db_plugin_manager()
plugins = mgr.list_plugins()
valid = mgr.validate_dsn("postgresql://user:pass@host:5432/db.schema")
probe = mgr.test_connection("sqlite:///local.db")
```

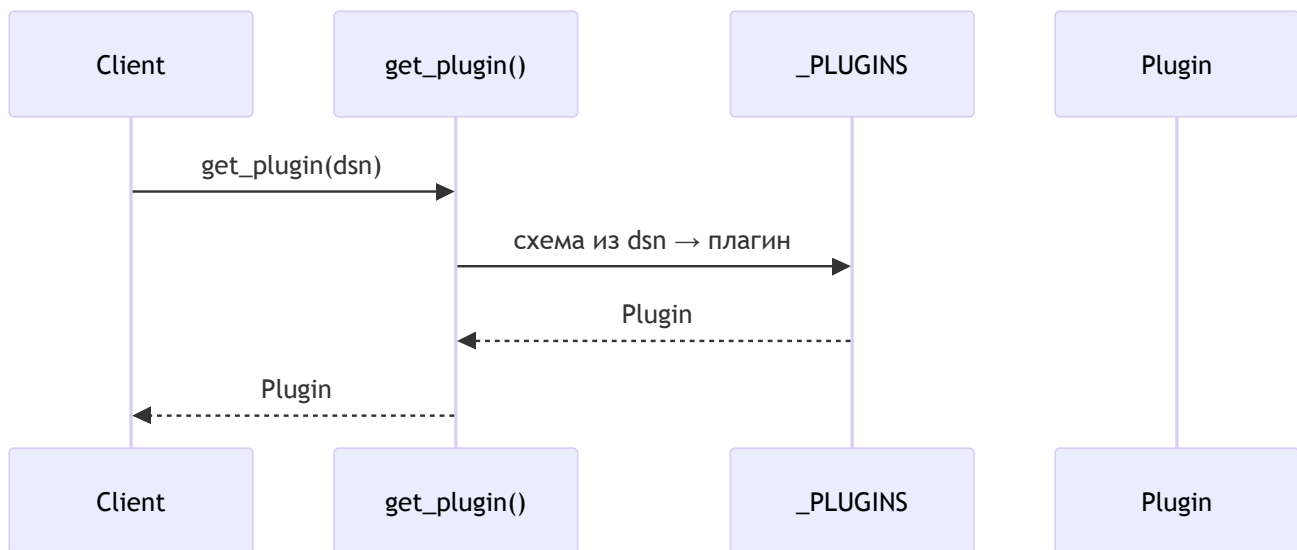
- list_plugins: какие диалекты доступны.
- validate_dsn: проверка схемы/формата.
- test_connection: реальный коннект + базовая информация о схеме.

Интроспекция (пример)

```
# SQLitePlugin.introspect_schema(conn)
cur.execute("SELECT name, sql FROM sqlite_master WHERE type='table'")
# ... парсинг и нормализация в стандартный формат ...
return standardized_schema
```

Плагин Postgres делает то же, но через information_schema.*

Диаграмма



Вывод

DBPluginManager скрывает различия СУБД и даёт единый интерфейс подключения, интроспекции и безопасного выполнения запросов, упрощая интеграцию в пайплайны.

Глава 19: Веб-интерфейс и Streamlit API

Стабильный слой-посредник между UI и ядром: запускает агентов и workflow, управляет процессами и возвращает статусы по типизированным контрактам.

Зачем

- Разделение ответственности: UI не знает внутренностей агентов.
- Стабильные контракты данных (dataclasses): совместимость при рефакторинге.
- Отзывчивость UI: выполнение в отдельных процессах.

Компоненты API

- AgentManager (agent_streamlit_api.py): запуск одиночных агентов/команд.
- WorkflowManager (workflow/streamlit_api.py): запуск YAML-пайплайнов.
- MemoryRAGManager (memory/streamlit_api.py): поиск/статусы памяти.
- DBPluginManager (db_plugins/streamlit_api.py): управление подключениями БД.

Пример: запуск агента

```
from agent_streamlit_api import AgentManager
mgr = AgentManager()
run_id = mgr.run_agent("DataAnalyst", task)
```

Под капотом:

```
def run_agent(self, agent_id_or_profile: str, task: str) -> str:
    run_id = f"run-{uuid.uuid4().hex[:16]}"
    proc = Process(target=_agent_process_entry, args=(run_id, ...))
    proc.start()
    self.active_runs[run_id] = {"status": "running", "pid": proc.pid}
    return run_id
```


Контракты статусов

```
@dataclass
class AgentRunStatus:
    run_id: str
    agent_name: str
    status: str # queued|running|completed|failed|cancelled
    task: str = ""
    start_time: datetime | None = None
```

UI получает предсказуемую структуру, независимо от изменений внутри менеджера.

Workflow из UI

```
from workflow.streamlit_api import WorkflowManager
wm = WorkflowManager()
run_id = wm.start_workflow("simple_research", parameters={"topic": topic})
```

Связка с остальными API

- `memory/streamlit_api.py` : поиск по RAG без знания векторного хранилища.
- `db_plugins/streamlit_api.py` : `list/validate/test` DSN.

Вывод

Streamlit API обеспечивает чистые контракты, изоляцию и масштабируемость интерфейса: UI обращается к менеджерам, а те — к ядру системы, сохраняя отзывчивость и устойчивость.

Глава 20: AgentManager для UI

Диспетчерский слой для запуска агентов из веб-интерфейса: изоляция процессов, стабильные API и отслеживание статусов по `run_id`.

Задача

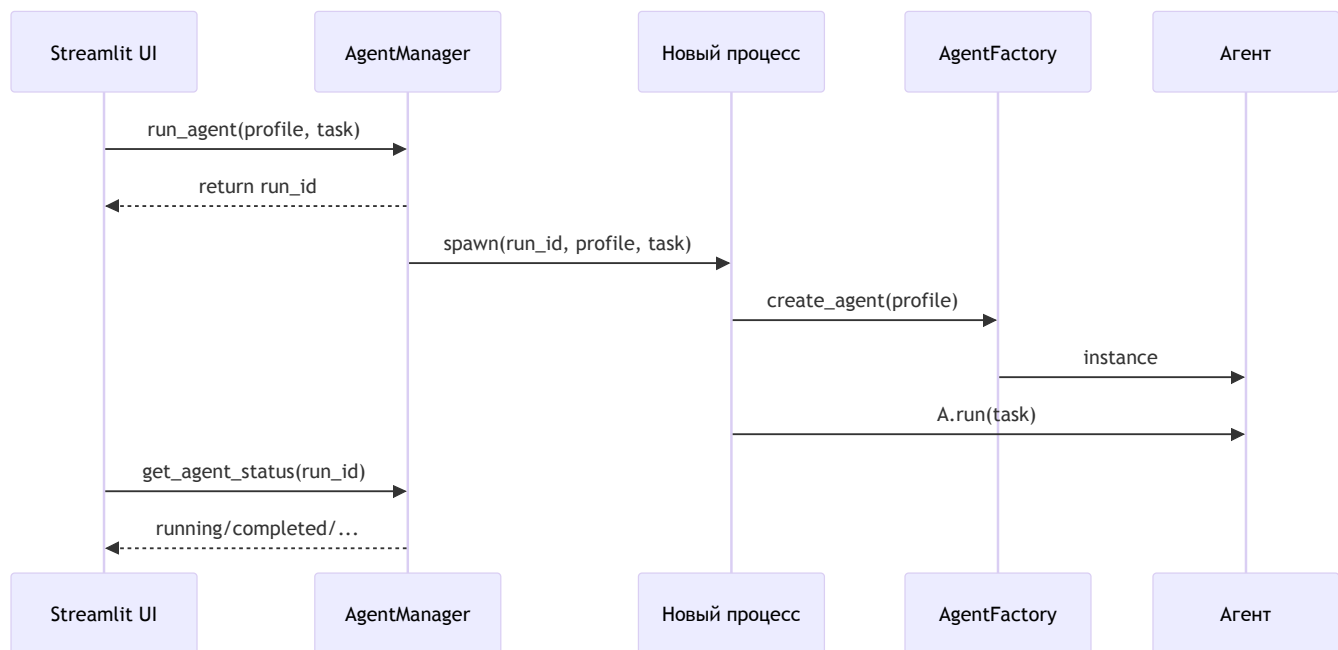
- Упростить UI: запускать агентов без знания внутренних деталей.
- Обеспечить устойчивость: отдельные процессы, отмена, статусы.

Основные операции

- `list_agents()` — список профилей для выбора в UI.
- `run_agent(profile, task) → run_id` — немедленный ответ, агент выполняется фонов.
- `get_agent_status(run_id)` — статус: `queued/running/completed/failed/cancelled`.
- `get_agent_result(run_id)` — итоговый отчет после завершения.

Как работает

- Каждый запуск — отдельный процесс с собственным агентом.
- Глобальный реестр `_GLOBAL_ACTIVE_RUNS` хранит статусы по `run_id`.
- Возврат `run_id` мгновенный; UI опрашивает статус отдельно.



Пример

```
mgr = AgentManager()  
run_id = mgr.run_agent("researcher", "Проанализируй рынок")  
status = mgr.get_agent_status(run_id)  
if status.status == "completed":  
    result = mgr.get_agent_result(run_id)
```

Итого

AgentManager обеспечивает чистый контракт для UI, изолируя тяжелую работу агентов и предоставляя управляемые статусы по run_id.

Глава 21: Менеджер Конфигурации (ConfigurationManager)

Централизованная конфигурация в YAML с горячим применением без перезапуска приложения.

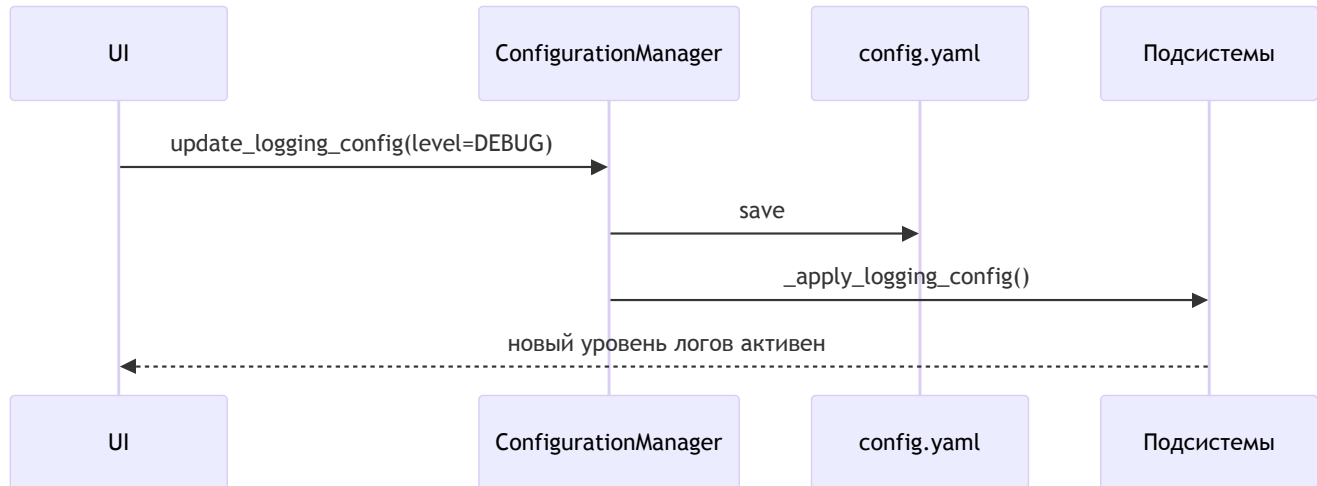
Зачем

- Единая точка для LLM, логов, телеметрии, безопасности.
- Правки из UI, моментальное применение.

Где хранится

- config/streamlit_config.yaml — человекочитаемый источник истины.
- Отражается в строго типизированных dataclass (например, SystemConfiguration).

Поток изменений



Интерфейс

```
cm = get_configuration_manager()
config = cm.get_config()
config.logging.level = "DEBUG"
cm.update_logging_config(config.logging)
```

Применение «на лету»

- `_apply_logging_config()` — меняет уровень root logger.
- Аналогично для телеметрии/LLM и др. через `_apply_*`.

Итого

ConfigurationManager отделяет код от настроек, ускоряет отладку и повышает управляемость системы.

Глава 22: Логирование (UnifiedLoggingManager)

Единая система логов, коррелируемых по `run_id` даже в отдельных процессах.

Проблема

Многопроцессность перемешивает сообщения; сложно отследить один запуск.

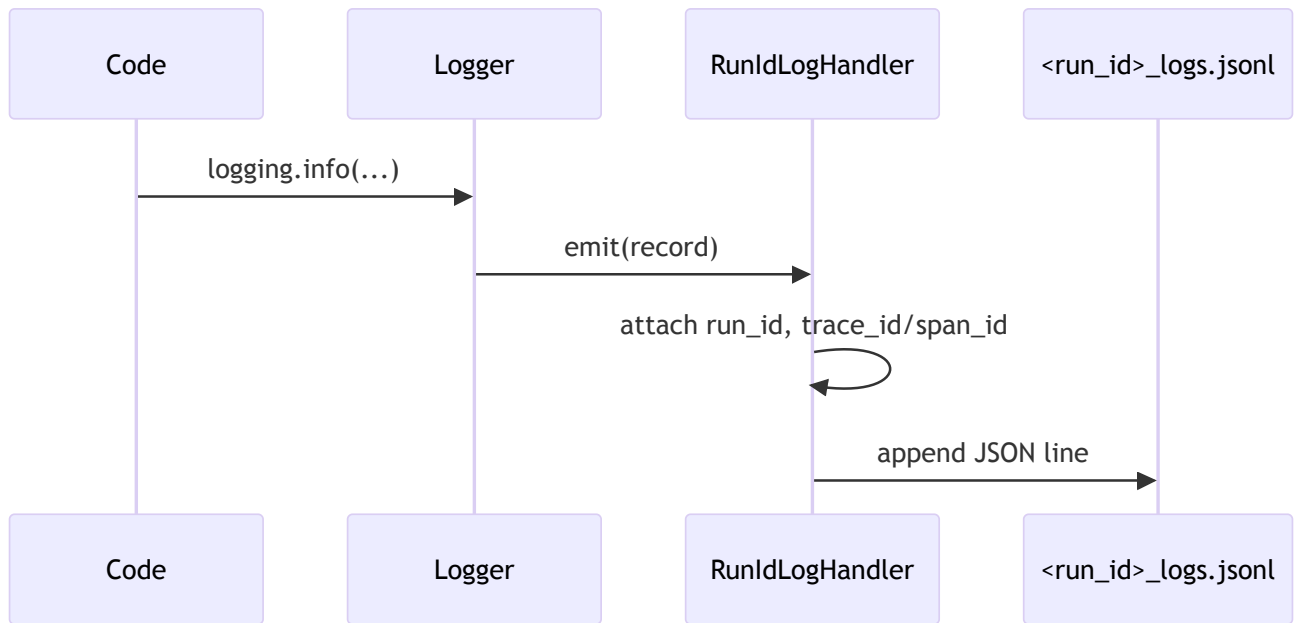
Решение

- `run_id_context(run_id)` — контекст, добавляющий `run_id` ко всем логам.
- Кастомный `RunIdLogHandler` — пишет JSONL в файлы вида `<run_id>_logs.jsonl`.
- Интеграция с OpenTelemetry: `trace_id`, `span_id` в каждой записи.

Использование

```
with run_id_context(run_id):  
    logger.info("Старт агента")  
    # ...  
    logger.info("Финиш")
```

Поток



Итого

Структурированные логи на запуск, готовые к анализу и связке с трассами телеметрии.

Глава 23: Телеметрия (SmolagentsTelemetryManager)

Глубокие трассы выполнения агентов и инструментов на базе OpenTelemetry с локальным хранением.

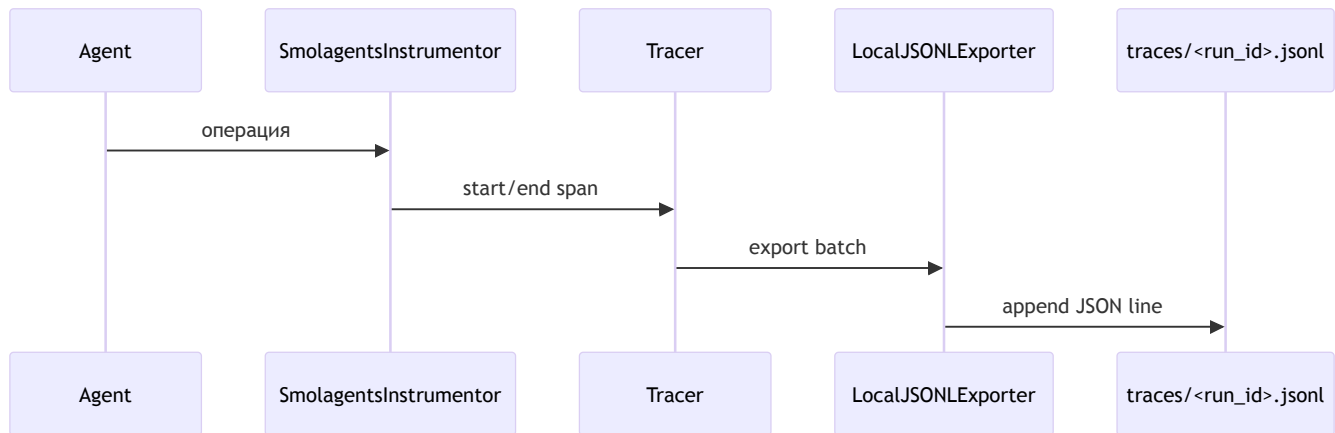
Зачем

- Видеть длительность и иерархию операций (спаны).
- Находить «узкие места» и коррелировать с логами по run_id.

Как устроено

- TracerProvider + SimpleSpanProcessor.
- LocalJSONExporter — пишет спаны в `<run_id>.jsonl`.
- RunIdPropagatingSpanProcessor — прокидывает `run_id` в каждый спан.
- SmolagentsInstrumentor — автотрейсинг вызовов `smol-agents`.

Поток



Просмотр в UI

- Страница Logs/Traces: список трасс, детальная диаграмма (Гант-подобная) по выбранному `run_id`.

Итого

Телеметрия делает выполнение прозрачным: где тратится время, какая операция «тормозит», и как она связана с логами.

Глава 24: Система устойчивости (Resilience)

Набор «систем безопасности» для предсказуемого и экономного выполнения: бюджеты, предохранители и детектор циклов.

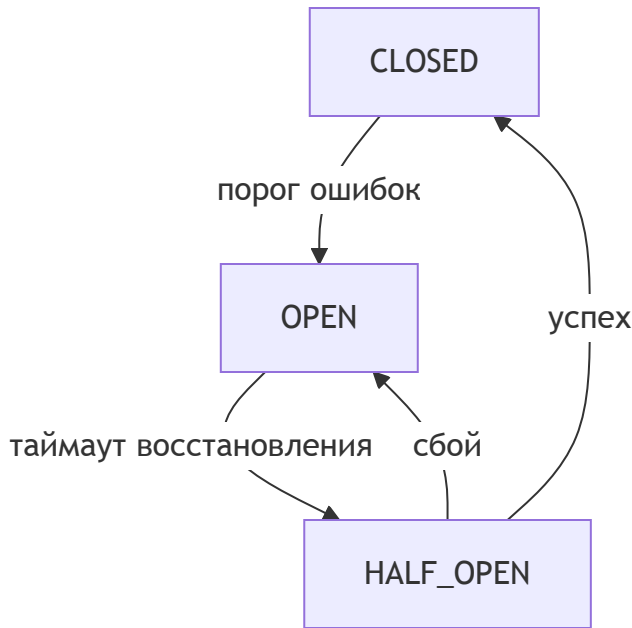
Компоненты

- BudgetManager: лимиты по времени/стоимости/токенам/API-вызовам.
- Circuit Breaker: быстрый отсекающий при лавине ошибок.
- Loop Detector: защита от закливания шагов.

BudgetManager

```
# Пример: задать лимиты и учитывать расход
budget_manager.create_workflow_budget(
    workflow_id="run-abc123",
    limits={"cost": 10.0, "time": 1800}
)
if budget_manager.check_budget("run-abc123", "cost", 0.05):
    # выполнить LLM-вызов и списать
    budget_manager.consume_budget("run-abc123", "cost", 0.05)
else:
    raise Exception("Бюджет исчерпан")
```


Circuit Breaker



```
# Безопасный вызов агента через предохранитель
result = await cb_manager.call_agent_safely(
    agent_name="researcher",
    agent_func=agent.run,
    task="Найди..."
)
```

Loop Detector

```
loop_detected = loop_detector.record_step_execution(
    workflow_id="run-abc123",
    step_id="research_step",
    execution_data={"task": "...", "output": "..."}
)
if loop_detected:
    raise RuntimeError("Обнаружено заикливание")
```

Итого

Resilience-слой предотвращает перерасход и простои: ограничивает бюджеты, гасит лавины ошибок и останавливает бесконечные повторы.

Глава 25: Механизм повторных попыток (Retry Engine)

Настойчивость при временных сбоях и адаптивные фоллбеки на альтернативные модели/источники.

Идея

- Повторные попытки с экспоненциальной задержкой и jitter.
- Классификация ошибок: временные (retry) vs фатальные (fallback/abort).
- Список fallback-целей (моделей) по приоритету.

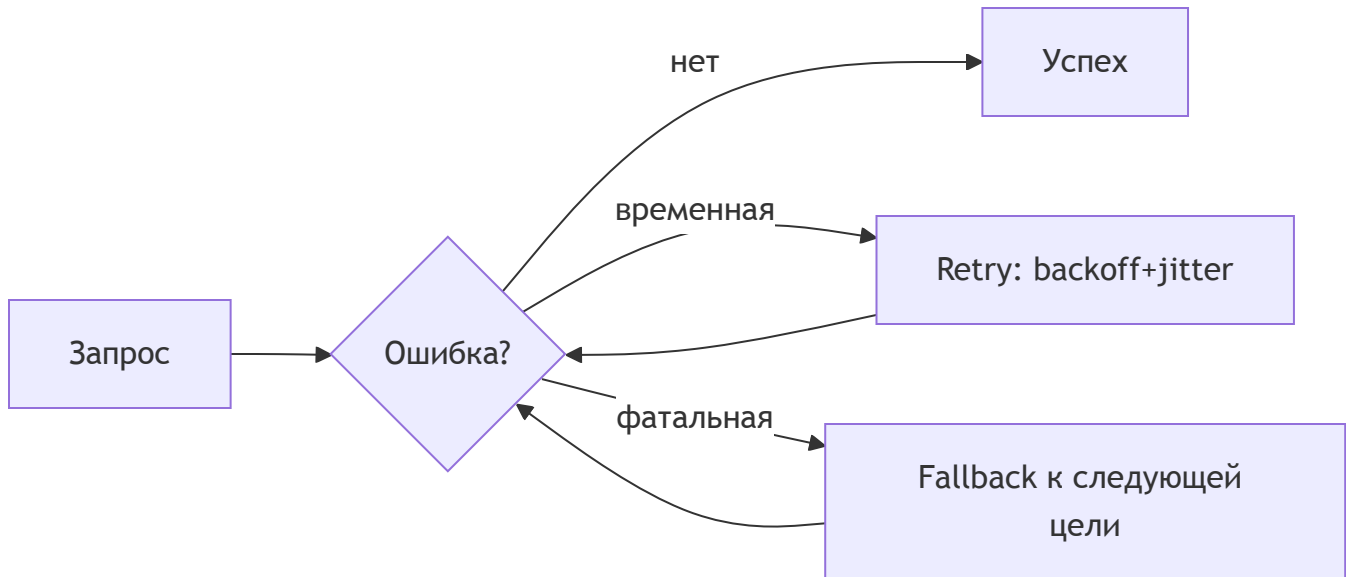
Классификация ошибок (пример)

- retry: 5xx, timeouts, connection reset.
- fallback: 404 model not found, quota exceeded, постоянный 429.

Псевдокод

```
def call_with_retry(send, fallbacks, max_retries):
    models = [primary] + fallbacks
    i = 0
    while i < len(models):
        for attempt in range(max_retries + 1):
            try:
                return send(models[i])
            except Exception as e:
                if should_fallback(e):
                    break # перейти к следующей цели
                sleep(backoff(attempt) + jitter())
        i += 1
    raise RuntimeError("All targets unavailable")
```

Диаграмма



Итого

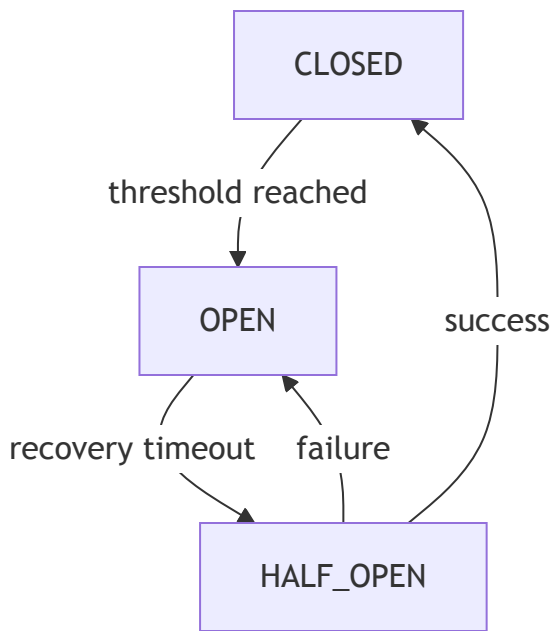
Retry Engine сглаживает сбои и автоматизирует переключение на альтернативы, повышая надежность без усложнения пользовательского кода.

Глава 26: Circuit Breaker

Предохранитель, предотвращающий лавину неудачных вызовов к сбойному агенту/сервису.

Состояния

- CLOSED: все вызовы проходят; считаем ошибки.
- OPEN: временная блокировка; новые вызовы отклоняются сразу.
- HALF-OPEN: пробный вызов после таймаута восстановления.



Безопасный вызов

```
result = await cb_manager.call_agent_safely(  
    agent_name="researcher",  
    agent_func=agent.run,  
    task="Найди свежие источники"  
)
```

Параметры

- `failure_threshold`: сколько подряд ошибок до OPEN.
- `recovery_timeout`: пауза перед HALF-OPEN.
- `error_classifier`: что считать «сбоем» для счетчика.

Итого

Circuit Breaker защищает систему от повторяющихся сбоев и экономит бюджет, быстро «отсекая» проблемный контур и позволяя ему восстановиться.

Глава 27: HTML Visualizer

Преобразует сырой результат агентов (Markdown/ссылки/диаграммы/изображения) в красивый интерактивный HTML-отчет.

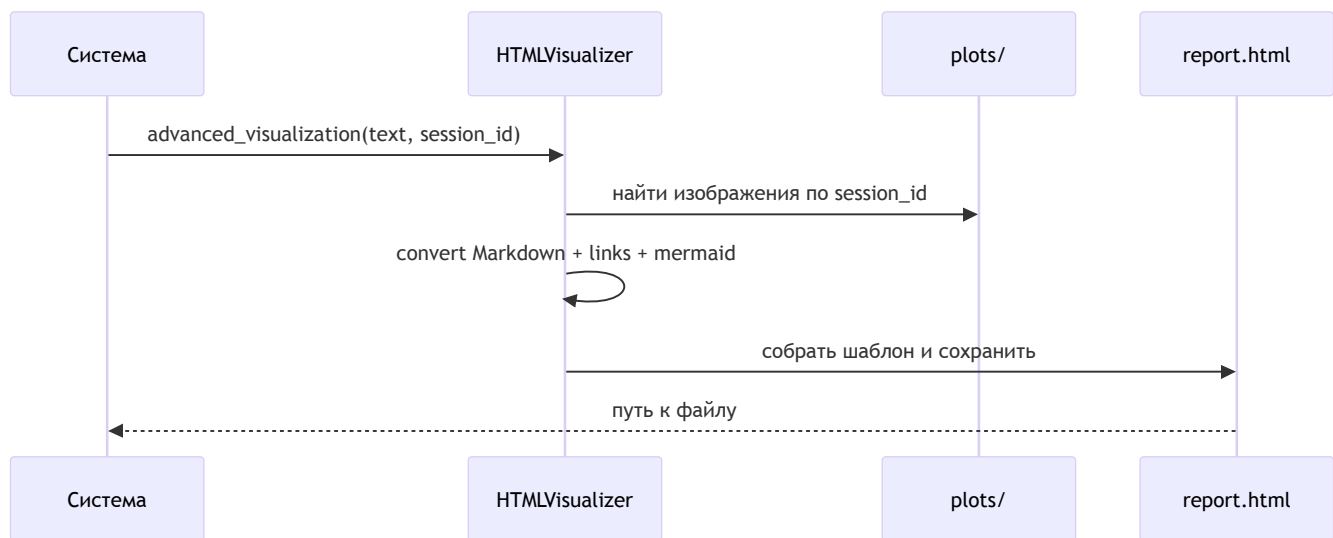
Что делает

- Конвертирует Markdown → HTML, ссылки → кликабельные.
- Встраивает PNG-графики по session_id (Base64).
- Поддерживает Mermaid-диаграммы с переключением темы, копированием кода и экспортом.

Использование

```
path = html_visualizer.advanced_visualization(result, session_id, show=True)
```

Конвейер



Итого

HTML Visualizer отвечает за «последнюю милю»: делает отчеты удобочитаемыми и интерактивными без ручной верстки.

Глава 28: Менеджер плагинов БД (DBPluginManager)

Расширенное администрирование плагинов и подключений: реестр, политики, здоровье и наблюдаемость.

Реестр и жизненный цикл

- Регистрация плагинов по схеме DSN в `_PLUGINS` .
- Версионирование/обновление плагинов через единый менеджер.
- Выгрузка/перезагрузка без перезапуска UI.

Политики

- Валидация DSN (обязательные компоненты: схема/хост/база/схема БД).
- Разрешения на диалекты (allowlist/denylist).
- Ограничения на операции (например, запрет `DELETE/UPDATE`).

Здоровье и кеш

- `test_connection(dsn)` — пробный коннект + легкий introspect.
- `health_status()` — периодическая проверка доступности источников.
- Кеш схем с TTL, инвалидация при изменениях.

Телеметрия

- Метрики подключений/ошибок по диалектам.
- Логирование попыток и результатов introspect/execute с `run_id`.

Интерфейс (пример)

```
mgr = get_db_plugin_manager()
for p in mgr.list_plugins():
    print(p.name, p.dialect)
assert mgr.validate_dsn(dsn).is_valid
probe = mgr.test_connection(dsn)
```

Итого

DBPluginManager централизует ответственность за поддерживаемые БД: стандартизирует регистрацию, контроль доступа и качество подключений, облегчая эксплуатацию и отладку.

Глава 29: Schema Linker

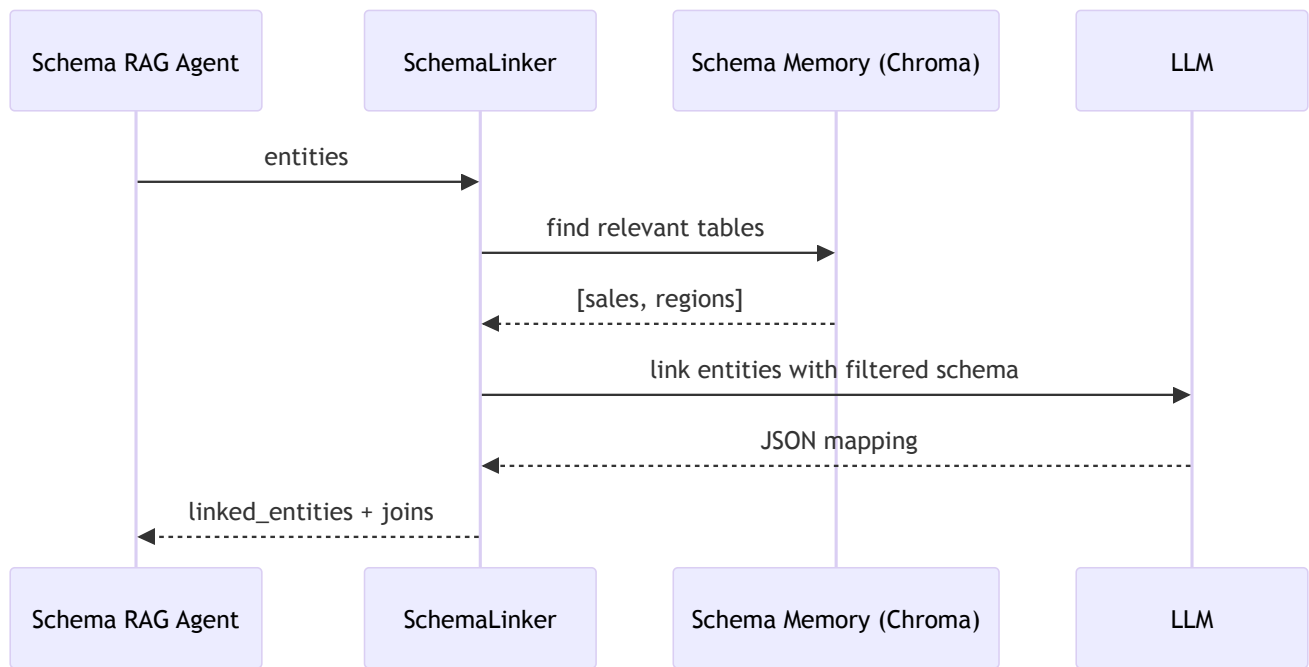
Картограф между естественным языком и схемой БД для Text-to-SQL.

Что делает

- Сопоставляет метрики/измерения/фильтры из NLU с таблицами и колонками.
- Строит необходимые JOIN-связи.

Гибридный подход

1. Семантический поиск по памяти схемы (RAG) → релевантные таблицы.
2. Сфокусированный промпт к LLM → точное сопоставление сущностей с полями.
3. Валидация + построение JOIN.



Пример результата

```
{
  "linked_entities": {
    "metrics": [{"name": "выручка", "table": "sales", "column": "amount"}],
    "dimensions": [{"name": "регионы", "table": "regions", "column": "region_name"}]
  },
  "joins": [{"from_table": "sales", "from_column": "region_id", "to_table": "regions",
}
```

Итого

Schema Linker превращает пользовательские сущности в конкретные поля БД, минимизируя контекст для LLM и обеспечивая точность и воспроизводимость.

Глава 30: RetryOpenAIServerModel

Надежный вызов LLM с повторными попытками и автоматическими фоллбеками на альтернативные модели.

Зачем

- Переживать rate limit/сети/временную недоступность.
- Прозрачно переключаться на запасные модели.

Инициализация

```
model = RetryOpenAIServerModel(  
    model_id="primary.model",  
    fallback_models="alt1.model,alt2.model",  
    max_retries=8,  
)
```

Алгоритм

- Для текущей модели: до `max_retries` попыток с паузами.
- На фатальные ошибки — переключение на следующую из `fallback_models`.
- Если все исчерпаны — исключение.

```
def __call__(self, messages, **kw):  
    while self.current_model_index < len(self.model_ids):  
        for attempt in range(self.max_retries + 1):  
            try:  
                return self.model(messages, **kw)  
            except Exception as e:  
                if self._should_fallback(e):  
                    break  
                time.sleep(self._backoff(attempt))  
        self._switch_to_fallback()  
    raise RuntimeError("All models unavailable")
```

Итого

`RetryOpenAIServerModel` повышает отказоустойчивость без усложнения кода агентов: один интерфейс — многие попытки и фоллбеки.

Глава 31: Система наблюдаемости (Observability)

Надсистема, объединяющая логи и телеметрию в связный взгляд на работу агентов.

Из чего состоит

- Логирование (см. 22): структурированные JSONL-логи с `run_id`, `trace_id`, `span_id`.
- Телеметрия (см. 23): трассы/спаны OpenTelemetry в локальных `<run_id>.jsonl`.

Корреляция

- Каждый лог-запись помечается текущим `span_id/trace_id`.
- В UI можно кликнуть на спан и увидеть только связанные логи.

Хранение и просмотр

- Локальные файлы в `logs/` для простого развёртывания.
- Страница Logs/Traces: список запусков и детальная диаграмма исполнения.

Польза

- Быстрая диагностика причинно-следственных цепочек.
- Поиск узких мест по длительности спанов.
- Аудит действий агентов и инструментов.

Итого

Observability превращает систему из «черного ящика» в прозрачный: что произошло, когда и почему — в одном месте.